Zero-touchday provisioning

Chaining TP-Link ZTP Vulnerabilities for Infiltrating Networks

Stanislav Dashevskyi, Francesco La Spina

black hat USA 2026

About us

Stanislav Dashevskyi
Principal Security Researcher

Francesco La Spina
Senior Security Researcher

FORESCOUT RESEARCH | VEDERE LABS

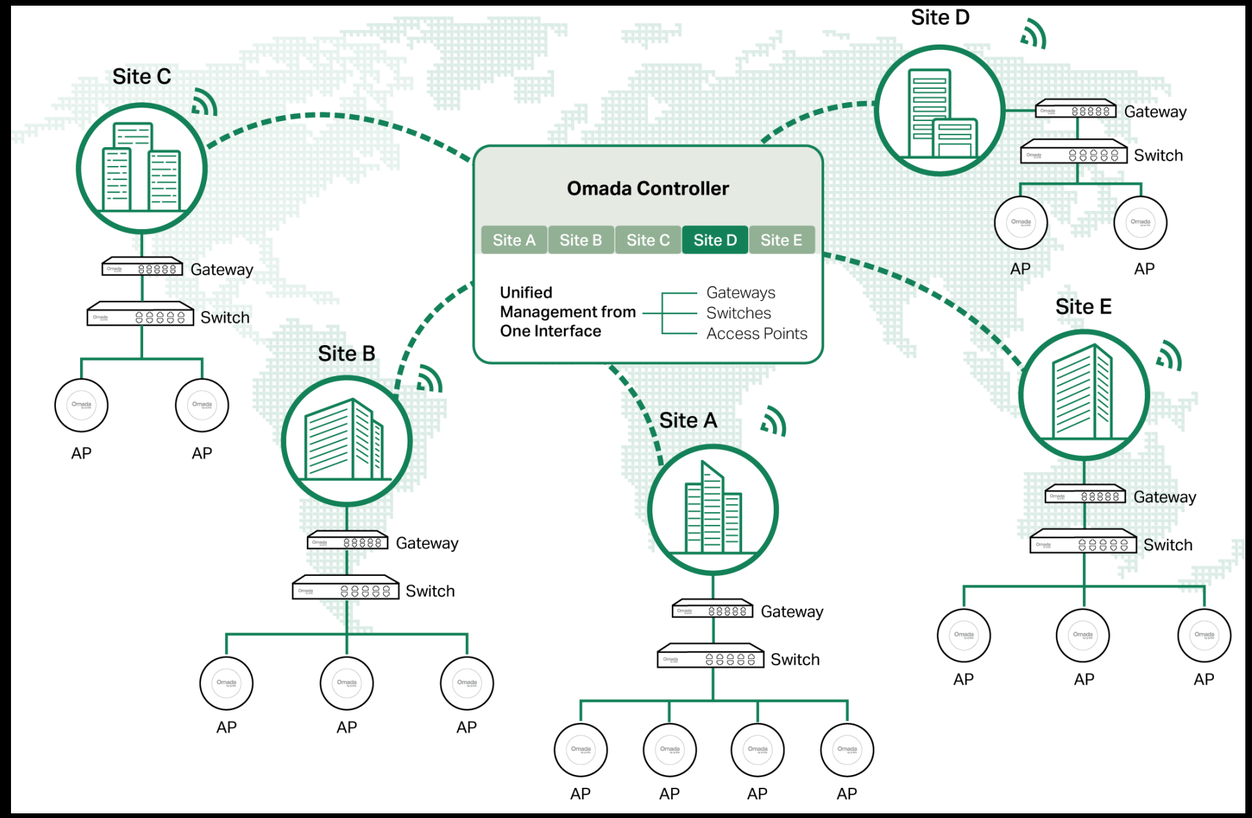
What is ZTP and why should we care?

From manual to “zero-touch” provisioning

- On-site installation required: new devices had to be physically configured by an IT technician, one by one
- Slow and error-prone deployments, limited scalability
- Vendors started to automate this process a decade ago…

Zero-Touch Provisioning (ZTP)

- ZTP is a technology allowing to remotely onboard and provision network devices from a central platform
- Several vendors have started to offer ZTP under different names and using different protocols
- A ZTP system has two main components:
  - **Client devices**: switches, routers, Wi-Fi access points, and gateways that need to be provisioned and managed
  - **Controllers**: dedicated appliances and/or software platforms that provision, configure, monitor, and update client devices



Taken from support.omadanetworks.com

What makes ZTP interesting for security research?

- Not a new concept, but not much previous research done
- It depends on the **strong chain of trust** between controllers and managed devices
  - Centralized management means a single point of failure
  - There is no universal set of ZTP protocols
  - ZTP controllers and devices are trusted by the internal network

Improved usability may compromise security

TP-Link Omada

Omada is a ZTP ecosystem from TP-Link, tailored for SMEs*

- **Cloud-based** (hosted by TP-Link)
- **Local (on premise) via dedicated hardware Controllers**
- **Local via software Controllers** (Virtual Machine)
- There can be hybrid deployments

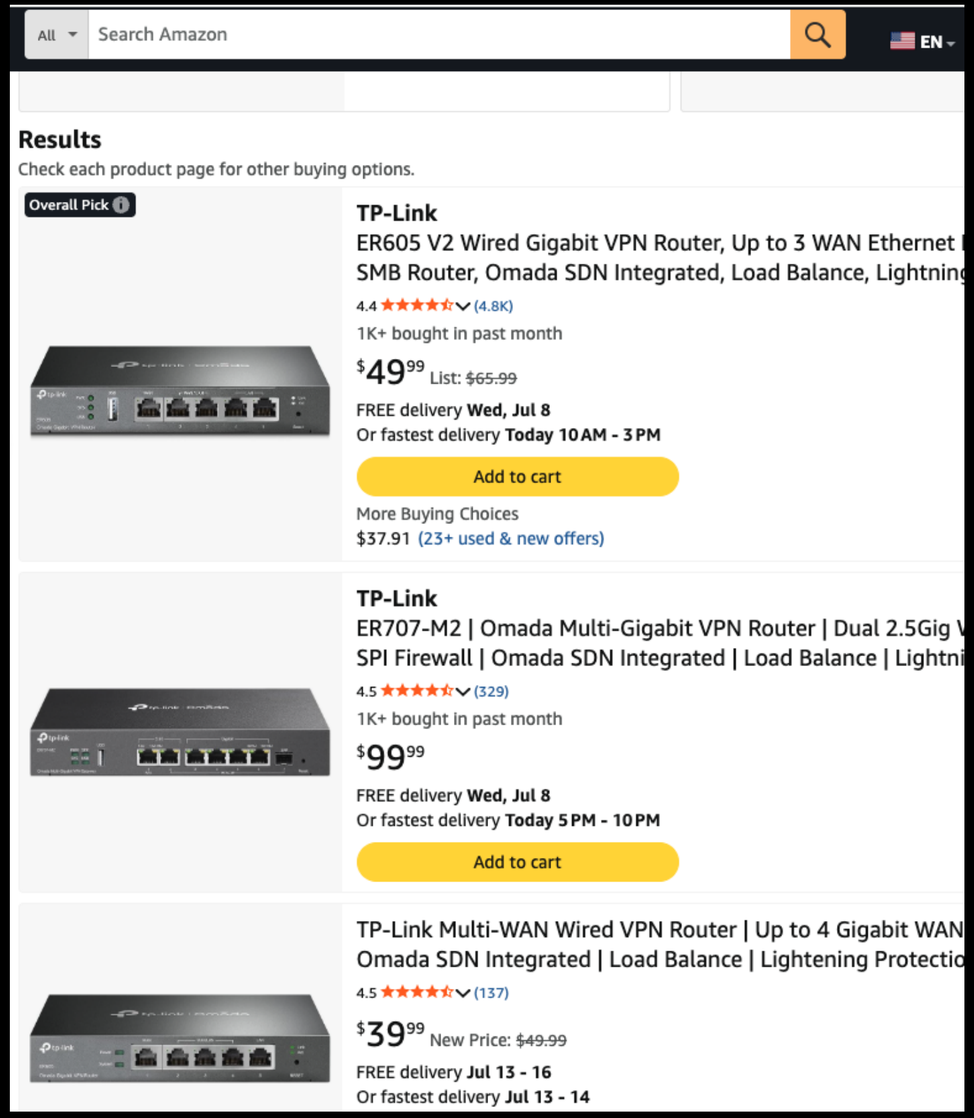
Taken from tplink.com

**Small and Medium Enterprises (SMEs)*

Why choosing Omada?

- **Large but under-researched attack surface**:
  - TP-Link has a large global deployment and customer base
  - Widely adopted ZTP ecosystem (SMB/SME) with limited public security research
  - Past vulnerabilities mostly on clients

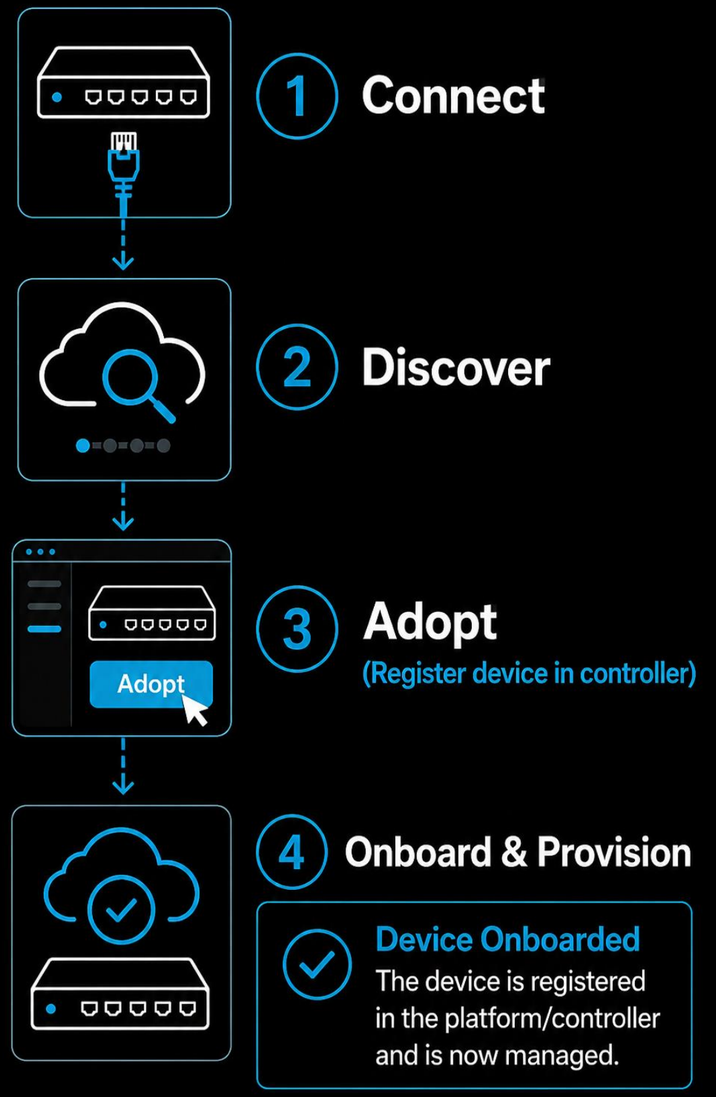
- **Rich feature-set for SME**
  - Overall, good hardware and functionalities
  - Easy to deploy and manage
  - Competitive value for money for SMEs (compared with Ubiquiti and Cisco)



What does ZTP look like in practice?

1. You connect a new Client device to the network
2. The Controller discovers the device
3. You click “Adopt” in the controller UI
4. The Controller adopts (onboards) and provisions the Client

- **But… how does this magic happen?**



Reverse-engineering Omada protocols

Where to begin? Client

- We purchased ER7206 and ER605 Omada gateways and downloaded the firmware
- We wanted to install debuggers and packet sniffers – **no root access!**
- Started with static analysis:
  - We looked at the Web UI of Client devices – it was built on **OpenWRT's LuCi**…
  - We had to figure out the way to **de-obfuscate** the **Lua bytecode** – tough!
- We found:
  - **CVE-2025-7851**: Insufficient patch for the "Leftover debug code" issue found by Cisco Talos (CVE-2024-21827)
  - **CVE-2025-7850**: Authenticated OS command injection via Wireguard VPN settings through the Web UI
- We had to release these findings as a separate blogpost

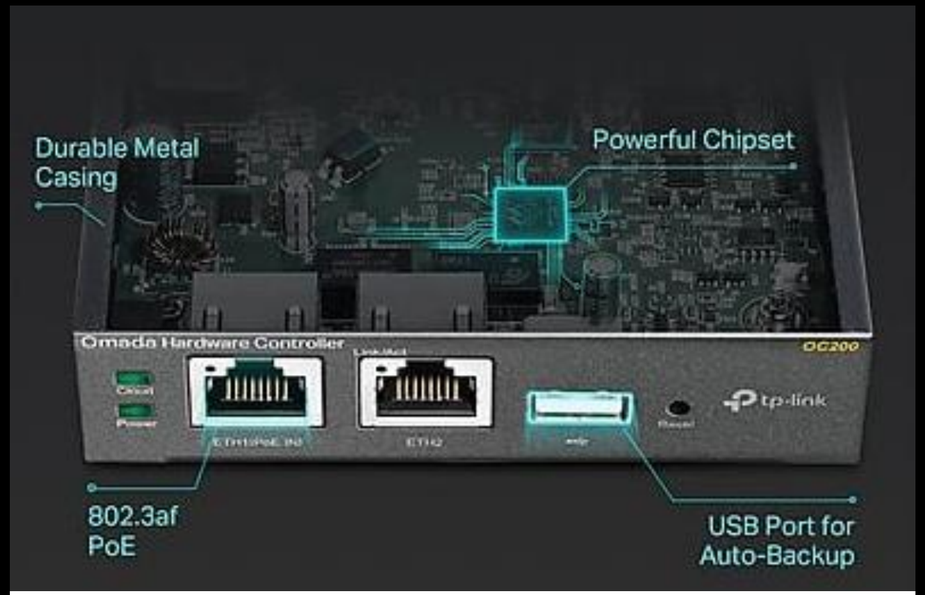
Taken from tplink.com

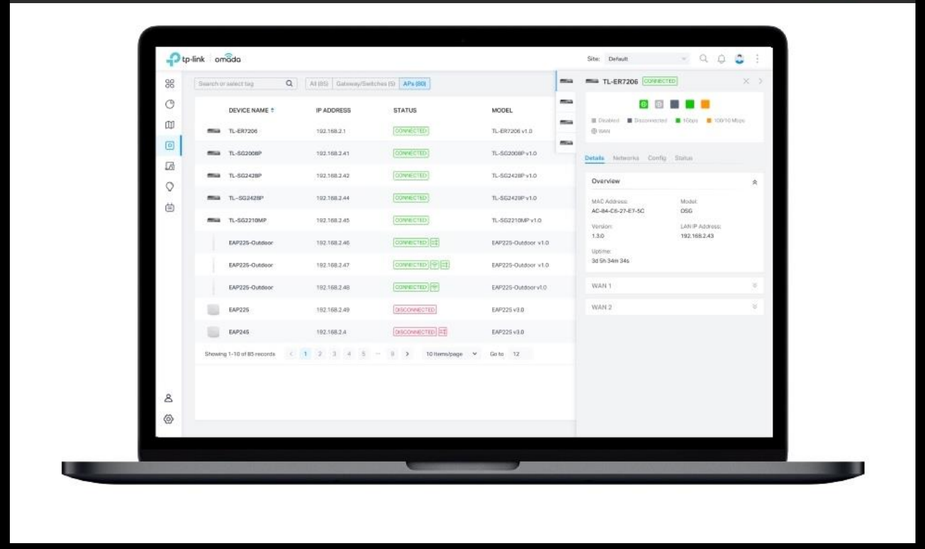
```
//...

if ( access("/usr/sbin/image_type_debug", 0) ) {
    // Send a challenge and verify the signature (requires the private key).
    // If the signature is correct, grant the root shell.
}
else {
    // Ask for a password derived from the LAN mac address.
    // If the password is correct, grant the root shell.
}
//...
```

Where to begin? Controller

- We purchased an **OC200** to use as a **local hardware controller**
- TP-Link also provides a **downloadable software version of the local controller** — Java based!
- Both helped us to analyse the communication protocol between clients and controllers and reverse-engineer the controller software.





Taken from tplink.com

Omada: Message format

- Several custom protocols:
  - UDP and TCP for transport, TLS for encryption
  - Each message is encoded in a JSON payload, preceded by a 4-byte payload length (network order)

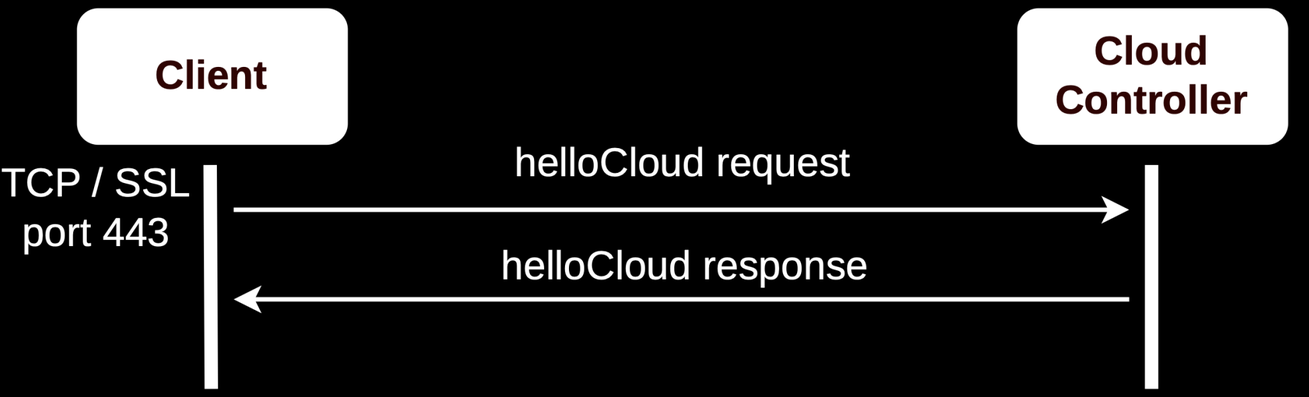
- Several phases: **Discovery, Adopt**, Manage, Reset, Prelink, Rebuilt, and others

- Similarities with different other protocols used by other (non-Omada) TP-Link devices
  - ***"I just wanted to learn the water temperature" by Imre Red, DeepSec'23***

Omada: Discovery (Local + Cloud)

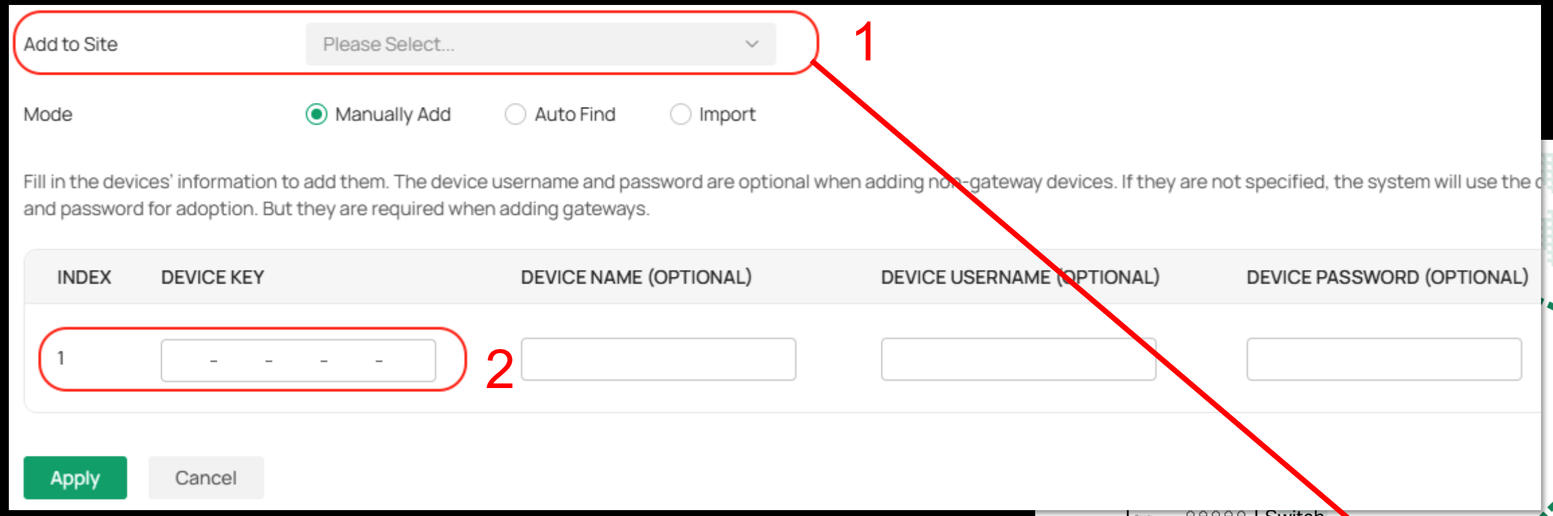
- Local discovery is done via UDP (broadcast)
  - Client sends basic information (S/N, MAC addr, model, version, etc.)
  - Controller responds with similar info + the IP/port to be used for Adoption

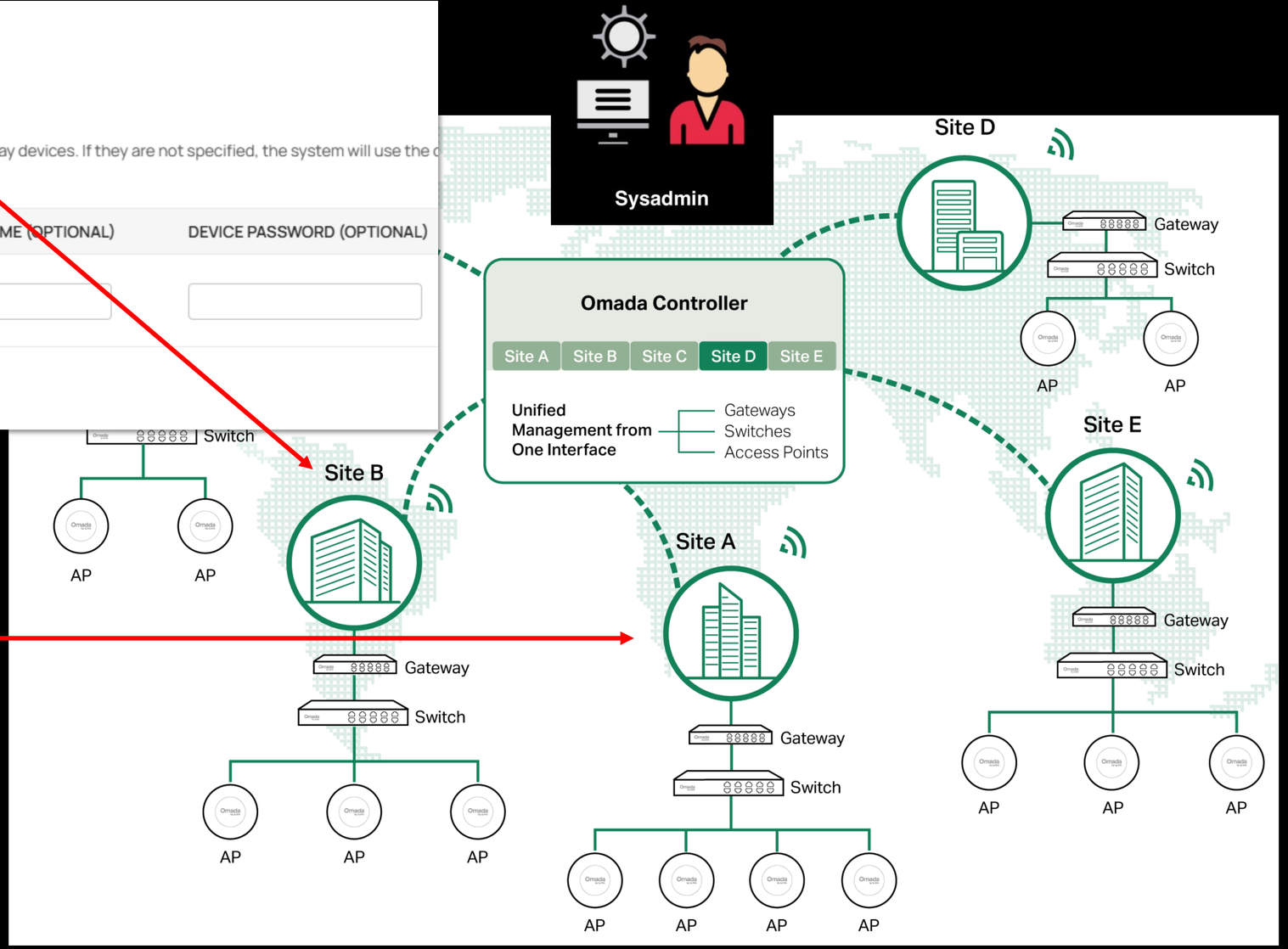
- Cloud-based discovery is via TCP/TLS
  - Similar info is exchanged, but the client will contact an Omada Cloud host



NOTE: When no local controller is present, Omada devices continue to broadcast Discovery protocol messages…**this will be relevant later**

Adoption steps (Cloud)





- Sites are logically separated network locations (different company branches)

Devices in each site will have shared "site credentials"

Taken from support.omadanetworks.com

Omada: Adoption V2

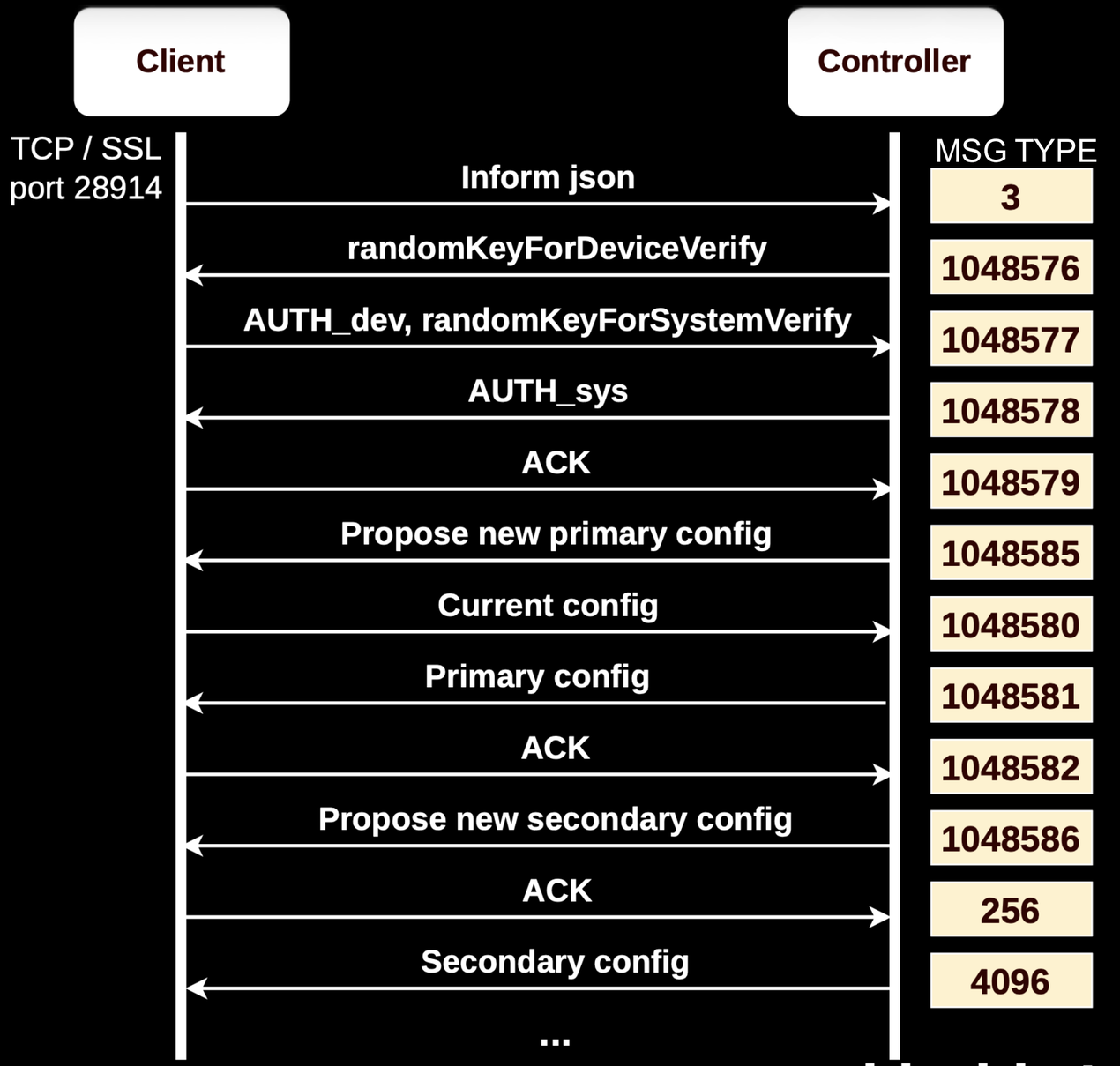
Works over TCP/TLS
No substantial differences between Local and Cloud

1. **During the TLS handshake, only** the client verifies the identity of the Controller

2. **Client authenticates with Controller (custom challenge-response)**
   a) Controller sends a “random” auth challenge
   b) Client replies with two-round hash of its creds, using the challenge as the salt (2nd round) -> no standard HMAC

3. **Controller authenticates with Client**
   a) Client sends its own “random” challenge
   b) Controller replies with two-round hash of device creds

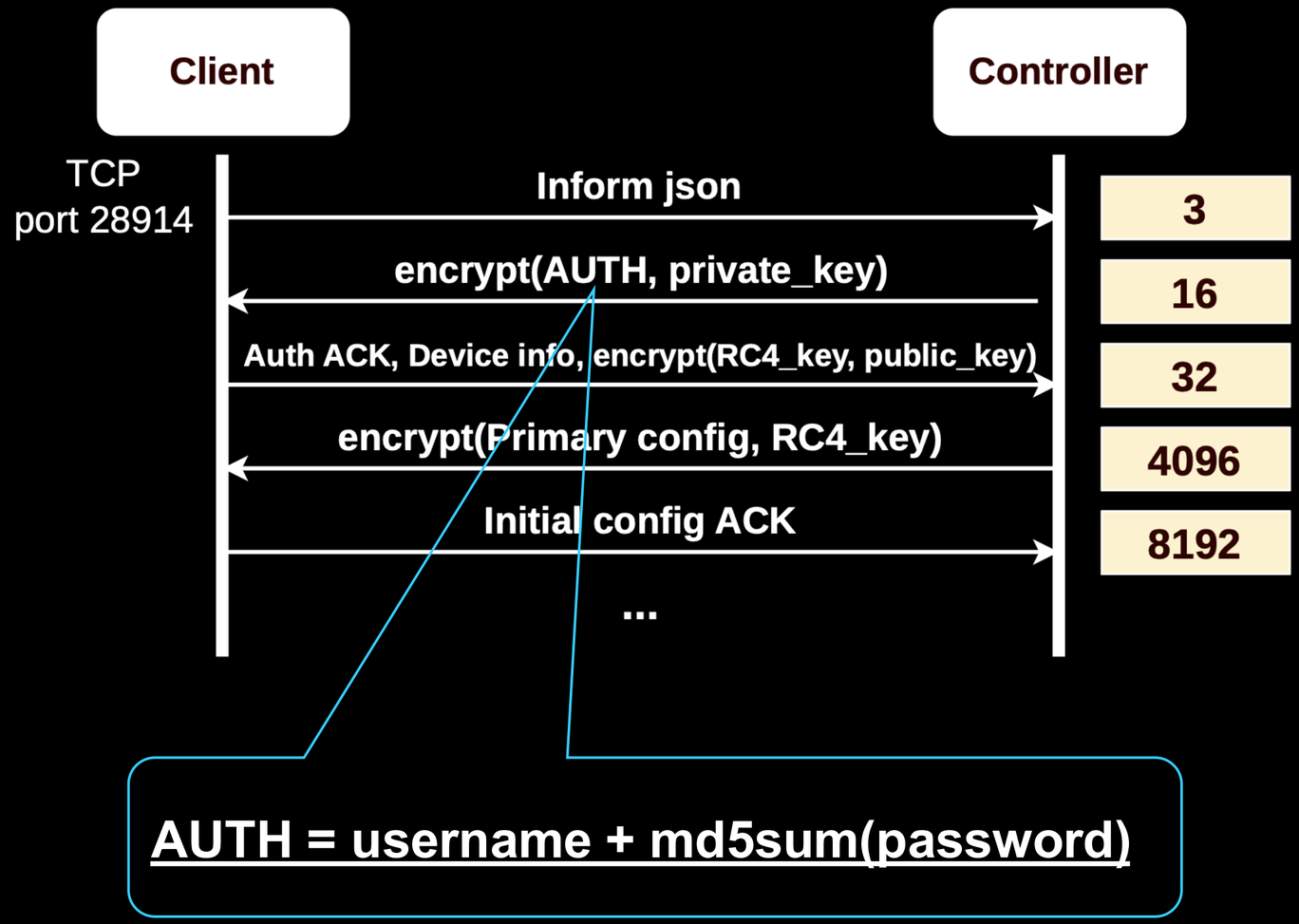
4. Next, Controller attempts to read the current config and push the new one
   a) Primary config: new network settings, “site credentials”…
   b) Secondary config: modules (Wireguard VPN) and other things
   c) Client is now adopted



The vulnerabilities

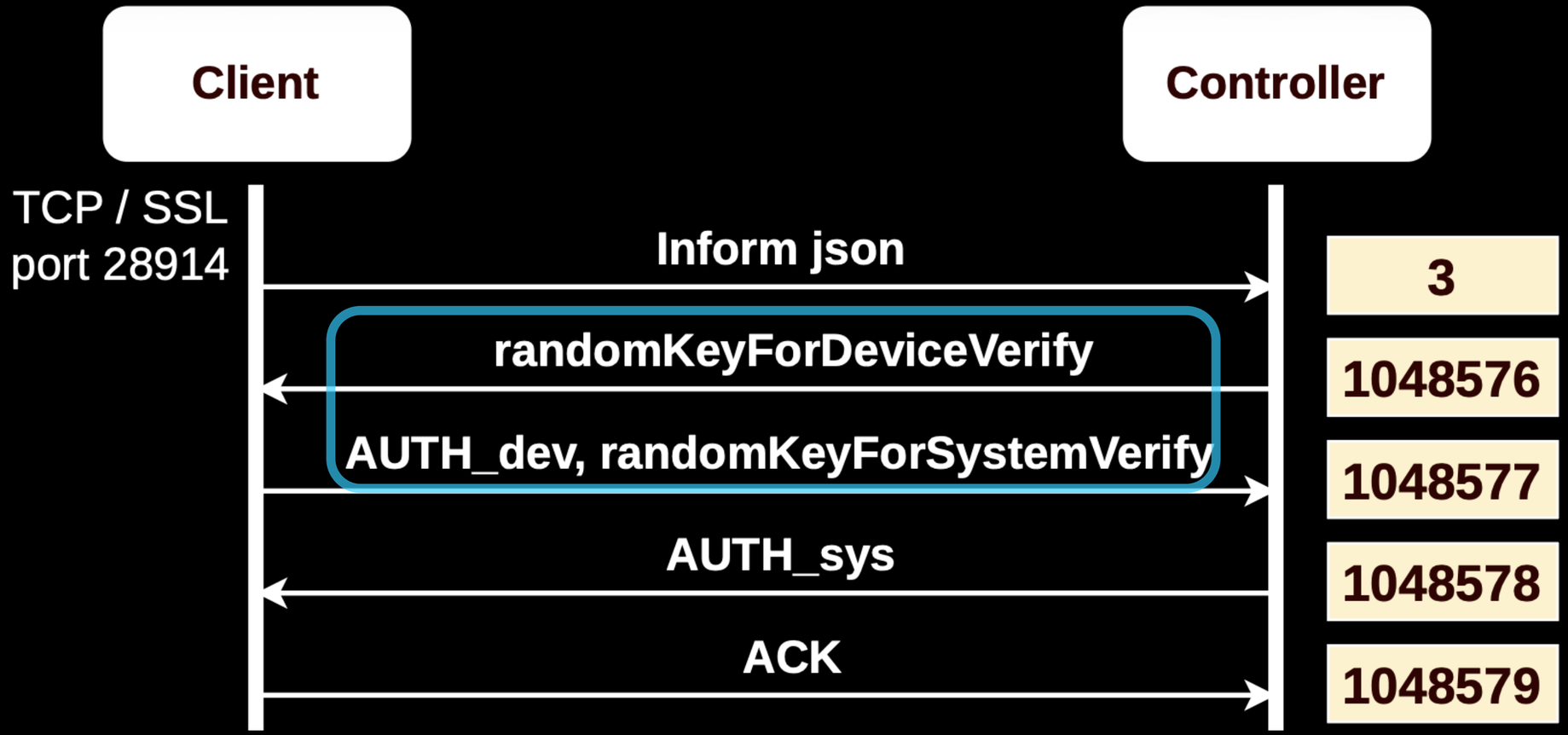
Hardcoded crypto in V1 (CVE-2025-15627)

- ”Legacy” V1 is present in the latest Clients, can be forced by Controllers
- Asymmetric encryption for auth and session key sharing, symmetric encryption for data
- The asymmetric keys are hardcoded
  - The “public” key is located in Client’s firmware
  - The private key is obfuscated within the Controller’s software
- **CVE-2025-15629**: the session key has insufficient entropy
- Attackers can decrypt traffic and get credentials



Insecure creds in V2 (CVE-2025-9290)

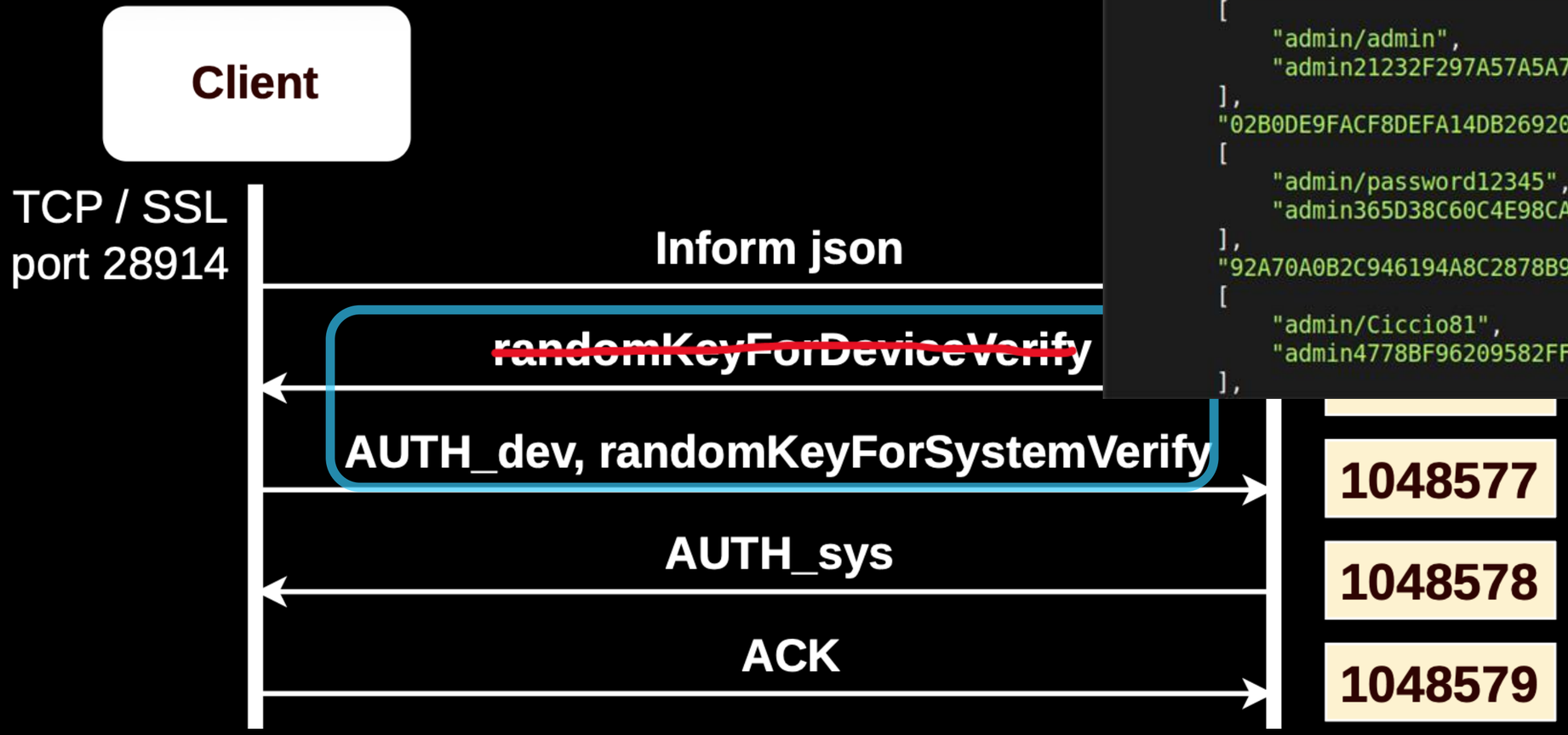
AUTH _dev = sha256sum(sha256sum(username + md5sum(password)) + randomKeyForDeviceVerify)



Insecure creds in V2 (CVE-2025-9290)

AUTH _dev = sha256sum(sha256sum(username + md5sum(password)) + ~~randomKeyForDeviceVerify~~)

```
rainbow_table = {
        "94F37F62C8CB0BF792518A11951EFC430620BB982C26454BD3230304157463E8":
        [
            "admin/admin",
            "admin21232F297A57A5A743894A0E4A801FC3",
        ],
        "02B0DE9FACF8DEFA14DB2692076FCF1B3A4E81D4A37B27E898A59CFC55CA462B":
        [
            "admin/password12345",
            "admin365D38C60C4E98CA5CA6DBC02D396E53",
        ],
        "92A70A0B2C946194A8C2878B9BAC3BB520A30CD41C4C44D7F7EC363F6F2EB2EB":
        [
            "admin/Ciccio81",
            "admin4778BF96209582FFA15C57F66BB00061",
        ],
```

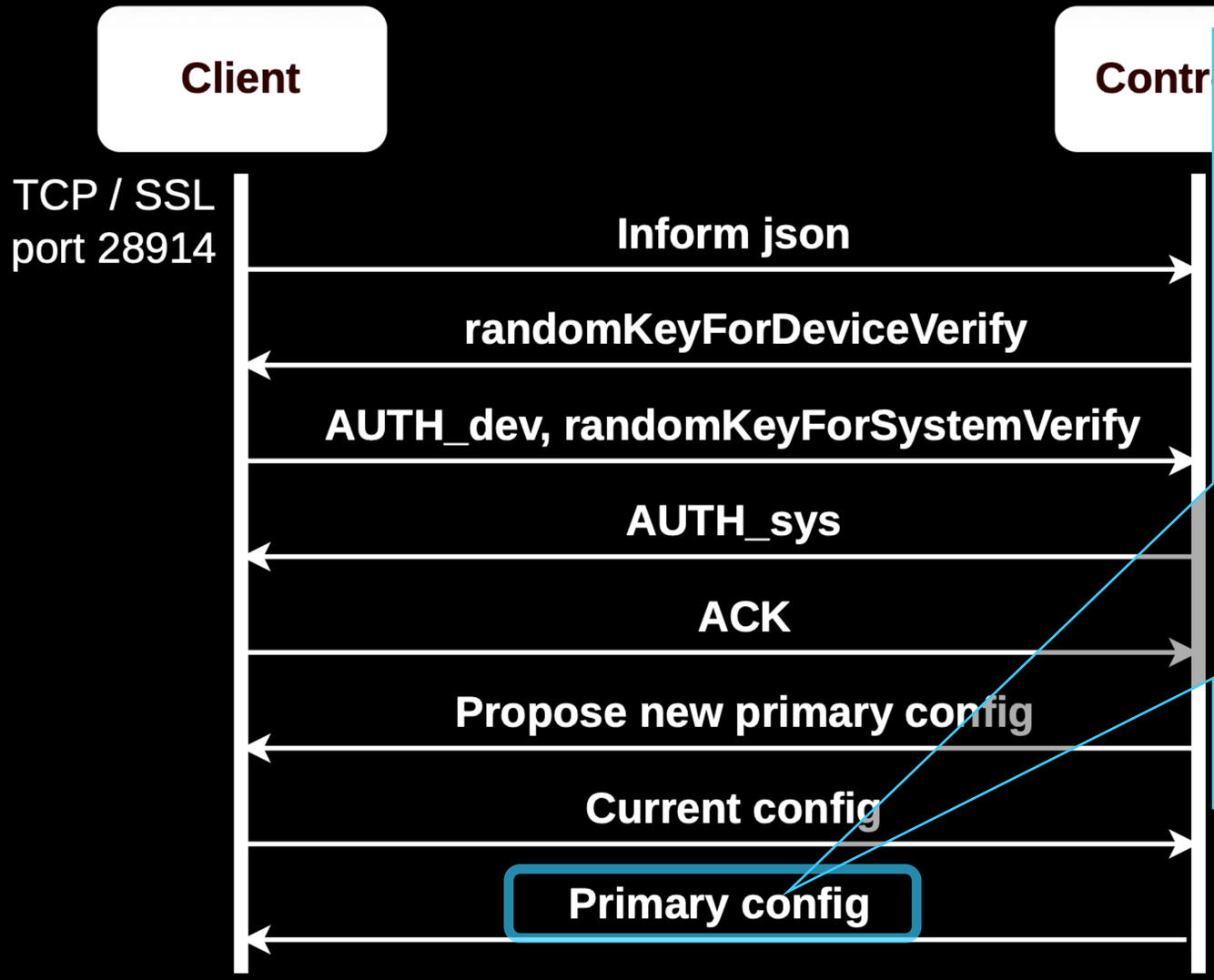


1048577

1048578

1048579

Insecure creds in V2 (CVE-2025-15544)

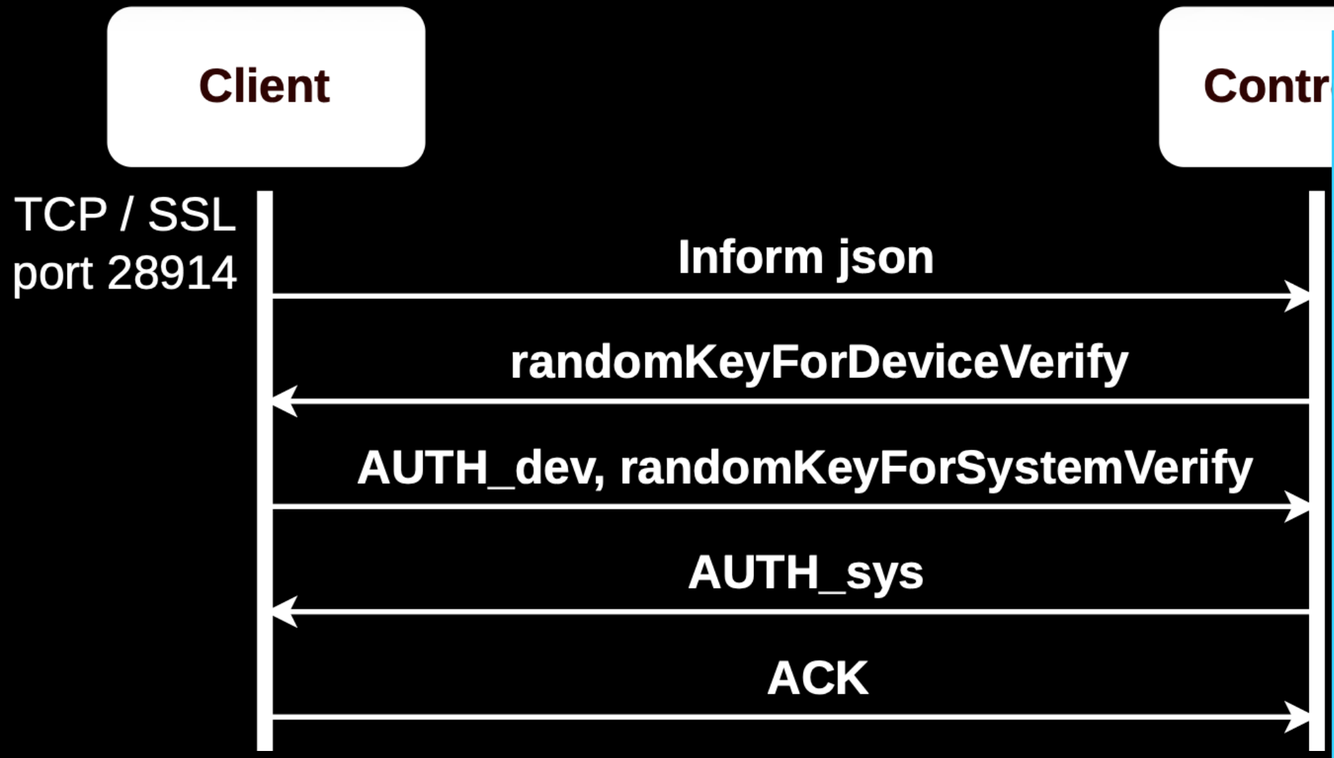



```
{
    "header" : {
        "error" : 0,
        "mac" : "00-DE-AD-BE-EF-00",
        "type" : 1048581,
        "version" : "2.2.0"
    },
    "body" : {
        // omitted for brevity
        "userAccount" : {
            "curPassword" : "21232F297A57A5A743894A0E4A801FC3",
            "curUsername" : "admin",
            "newPassword" : "A601EA74981ADF5A62C23DDA0E0D64CB",
            "newUsername" : "site_admin"
        }
    }
}
```

1048580

1048581

Insecure creds in V2 (CVE-2025-15544)



Controll

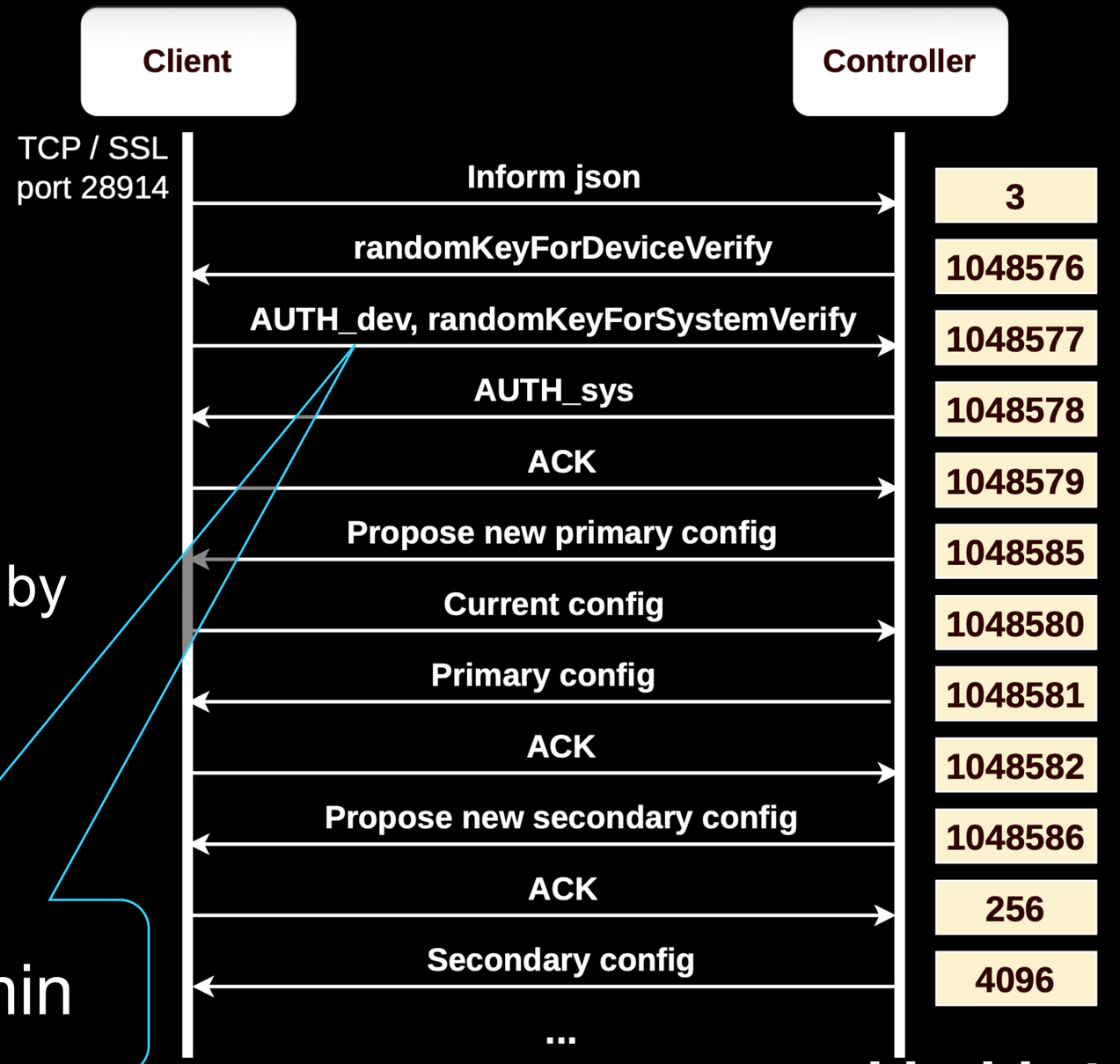
Controllers do not validate the identify of Clients

```
{
    "userAccount" : {
        "curPassword" : "21232F297A57A5A743894A0E4A801FC3",
        "curUsername" : "admin",
        "newPassword" : "A601EA74981ADF5A62C23DDA0E0D64CB",
        "newUsername" : "site_admin"
    }
}
```

AUTH _dev = sha256sum(sha256sum(username + md5sum(password)) + randomKeyForDeviceVerify)

Default credentials (FSCT-2025-008)

- A brand-new Client will always have default credentials (admin/admin)
- If we ”fake” a Controller, we can always know if a Client has default credentials
- If we ”fake” a Client and it gets accepted by a Controller, we can always authenticate
- **<u>You get site credentials “for free"</u>**

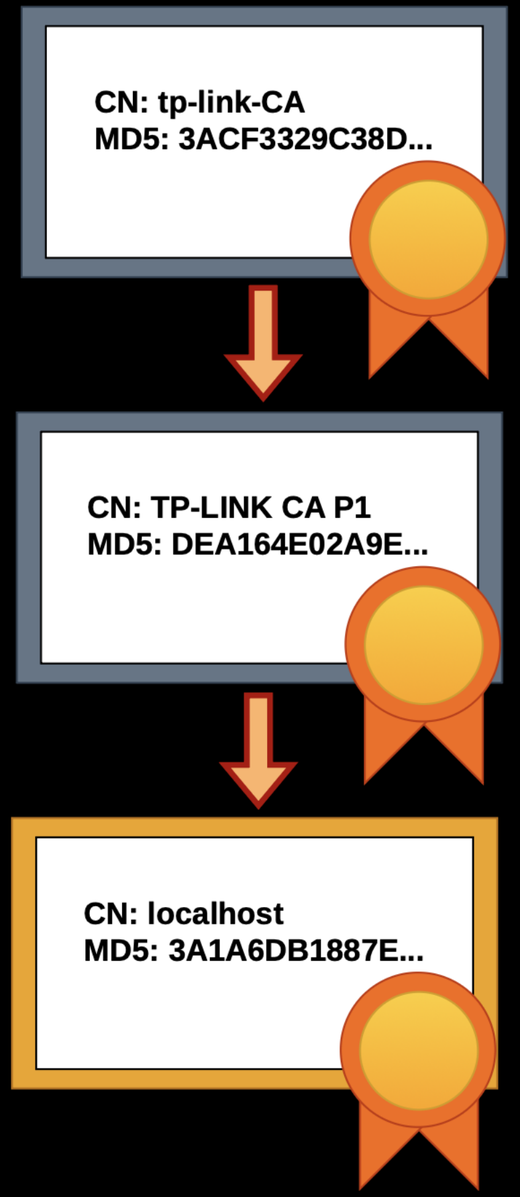


admin / admin

The chain of trust

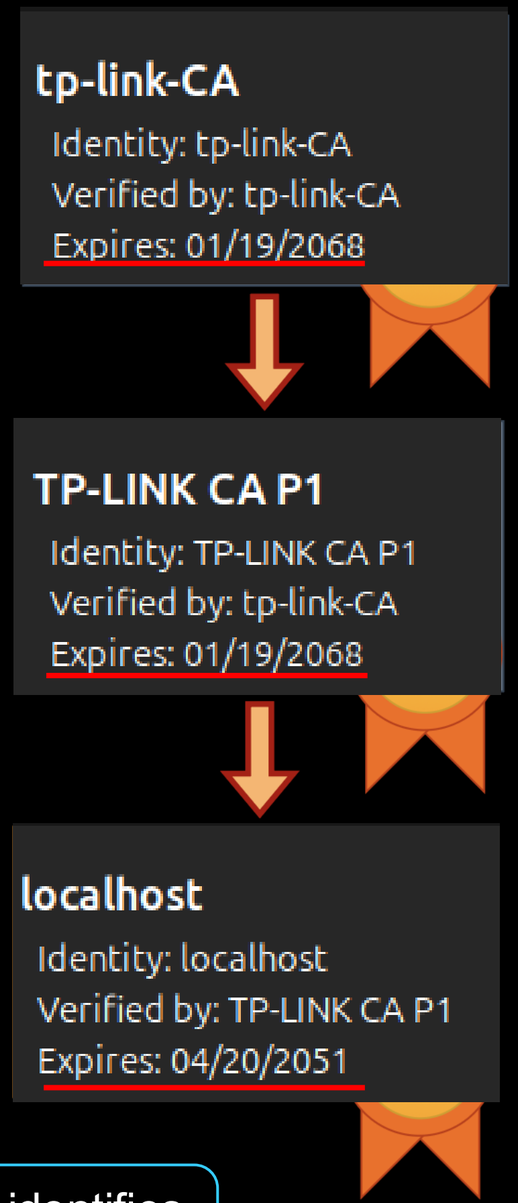
- The traffic in V2 is encrypted and Clients verify the identity of Controllers (TLS)

- Typical PKI chain of trust:
  - The **“Root” cert** is self signed and used as Root CA, imported into the keystore of Clients
  - The **“Intermediate” cert** is signed by the Root and is imported into the Controllers’ trusted keystore
  - The **”Server” cert** is used by Controllers and Clients for TLS



The broken chain of trust (CVE-2025-15628)

- Look at the cert expiration dates... Everything is hard-coded
- Controllers have a hard-coded private key (!) used for TLS (!)
- Controllers' Server cert is enough to pass the identity check -> **we can reuse Controller's Server certs and private key!**
- Cloud Controllers must present a cert derived from Root/Intermediate with a CN* that ends with "tplinkcloud.com" -> we cannot forge this one ☹



* A certificate common name (CN) is a standard field in an X.509 digital certificate that identifies the primary hostname, domain name, or entity protected by or assigned to the certificate

Why impersonating Controllers?

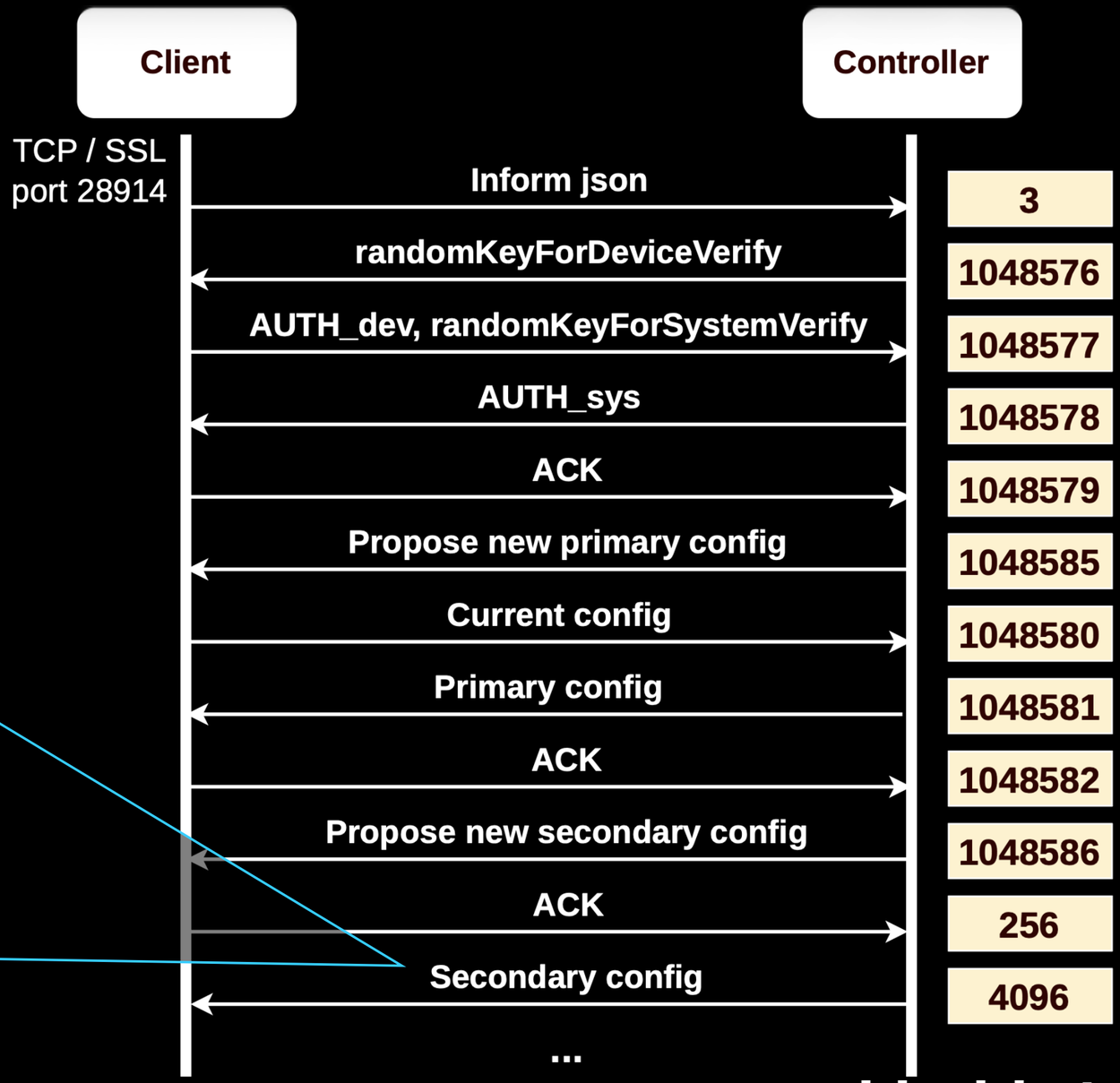
- Use Clients as credential oracles
- Get private VPN keys, read other sensitive information, push new settings
- Also, remember CVE-2025-7850?
  - OS command injection via the Web UI (authenticated!)

```
209   function check_key(r0_14, r1_14)
210     -- line: [226, 241] id: 14
211     local r2_14 = "echo \"" .. r9_0.safeSpecialString(r0_14) .. "\" |wg pubkey"
212     if r4_0.fork_call(r2_14) == 1 then
213       return true
214     end
215     if r1_14 then
216       r1_14.public_key = io.popen(r2_14):read("*a")
217     end
218     return false
219   end
```

```
echo "KPzPGI6WQ7z6bfwKYrQtpwRQwzgaU8/6wVEUtRIlslI="\n{ARBITRARY_OS_COMMAND}"" | wg pubkey
```

Using fake Controllers to exploit CVE-2025-7850

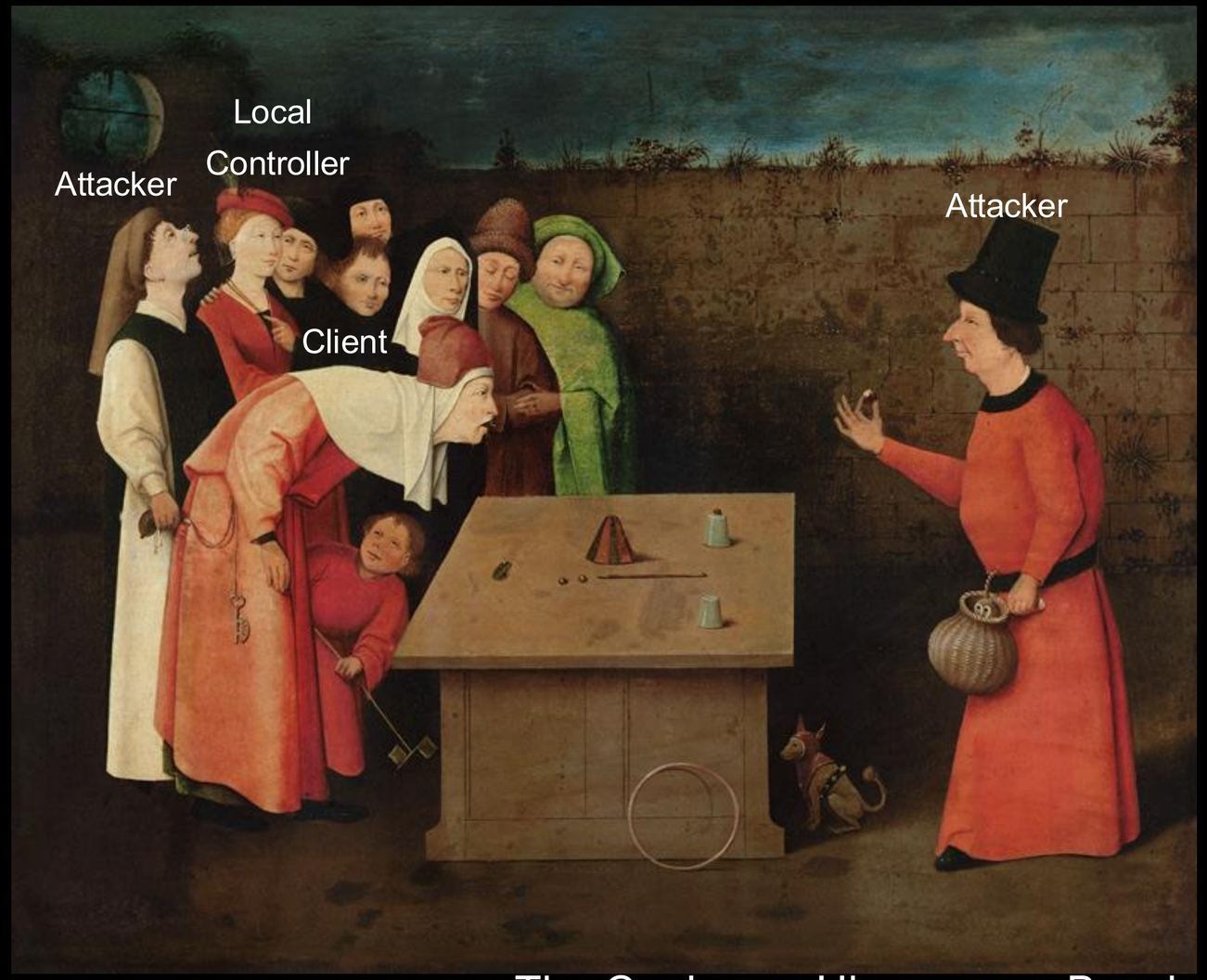
```
manage_config_msg = {
    "header" : {
        "type" : 4096,
    },
    "body" : {
       "sequenceId": 4,
       "userAccount": {
           # ...
       },
       "wireguard": {
           "interfaces": [
               {
                   "id":878491625,
                   "operation":1,
                   "enable":"true",
                   "mtu":1420,
                   "listenPort":51820,
                   "privateKey":"[ARBITRARY OS COMMAND]",
                   "localIp":"10.10.10.1"
               }
           ]
       },
       "configVersionInc": 1,
    }
}
```



Let’s catch our breath

- We can impersonate Clients and Local Controllers
  - RCE on unconfigured Clients, pivot to the entire site
  - Get site credentials from Clients/Controllers (also with passive traffic analysis)
  - Authenticate -> RCE on Clients

- **What can we do to Controllers?**
- **What about Cloud-based provisioning?**



The Conjurer - Hieronymus Bosch

The broken chain of trust — Cloud (CVE-2025-9291)

```json
{
   "id" : 1,
   "method" : "helloCloud",
   "params" : {
      "alias" : "ER605",
      "authCode" : "[REDACTED]",
      "cloudUserName" : "",
      "controllerVersion" : "",
      "deviceHwVer" : "2.0",
      "deviceId" : "[REDACTED]",
      "deviceMac" : "[REDACTED]",
      "deviceModel" : "ER605",
      "deviceName" : "ER605",
      "deviceType" : "SMBROUTER",
      "fwId" : "",
      "fwVer" : "2.2.6 Build 20240718 Rel.82712",
      "hwId" : "[REDACTED]",
      "oemId" : "[REDACTED]",
      "tcspVer" : "1.2"
   }
}

{
   "error_code" : 0,
   "id" : 1,
   "result" : {
      "cachedSvr" : "n-euw1-device-omada.tplinkcloud.com:443",
      "illegalType" : 0,
      "validTimeOnDevice" : 86400
   }
}
```

```
__int64 ecs_verifySsl(unsigned int preverify_ok, void* store_ctx) {
    // ...
01:   if ( !error_depth )
02:    {
03:      subject_name_str = strstr(subject_name_buf, "/CN=");
04:      if ( !subject_name_str )
05:        return 0;
06:      cert_subject_name = subject_name_str + 4;
07:      v11 = strchr(subject_name_str + 4, '/');
08:      if ( v11 )
09:        *v11 = 0;
10:      if ( strstr(global_controller_host, "tplinkcloud.com") && !strstr(cert_subject_name, "tplinkcloud.com") )
11:      {
12:        if ( HIDWORD(qword_5CCB8) )
13:          printf(
14:            "[ECS][ERROR]%s():%5d @ verify error:CN mismatch(%s), controllerUrl(%s).\n\r",
15:            "_ecs_verifySsl",
16:            125LL,
17:            cert_subject_name,
18:            global_controller_host);
19:        if ( qword_5CCB8 )
20:        {
21:          // ...
22:          ecs_log(2LL, "[ECS][ERROR]<%s>%s():%5d @ verify error:CN mismatch(%s), controllerUrl(%s).\n\r");
23:          // ...
24:        }
25:        return 0;
26:      }
27:    }
    // ...
  }
```

The broken chain of trust — Cloud (CVE-2025-9291)

```
{
   "id" : 1,
   "method" : "helloCloud",
   "params" : {
      "alias" : "ER605",
      "authCode" : "[REDACTED]",
      "cloudUserName" : "",
      "controllerVersion" : "",
      "deviceHwVer" : "2.0",
      "deviceId" : "[REDACTED]",
      "deviceMac" : "[REDACTED]",
      "deviceModel" : "ER605",
      "deviceName" : "ER605",
      "deviceType" : "SMBROUTER",
      "fwId" : "",
      "fwVer" : "2.2.6 Build 20240718 Rel.82712",
      "hwId" : "[REDACTED]",
      "oemId" : "[REDACTED]",
      "tcspVer" : "1.2"
   }
}

{
   "error_code" : 0,
   "id" : 1,
   "result" : {
      "cachedSvr" : "n-euw1-device-omada.tplinkcloud.com:443",
      "illegalType" : 0,
      "validTimeOnDevice" : 86400
   }
}
```

```
__int64 ecs_verifySsl(unsigned int preverify_ok, void* store_ctx) {
    // ...
01:   if ( !error_depth )
02:    {
03:      subject_name_str = strstr(subject_name_buf, "/CN=");
04:      if ( !subject_name_str )
05:        return 0;
06:      cert_subject_name = subject_name_str + 4;
07:      v11 = strchr(subject_name_str + 4, '/');
08:      if ( v11 )
09:        *v11 = 0;
10:      if ( strstr(global_controller_host, "tplinkcloud.com") && !strstr(cert_subject_name, "tplinkcloud.com") )
11:      {
12:        if ( HIDWORD(qword_5CCB8) )
13:          printf(
14:            "[ECS][ERROR]%s():%5d @ verify error:CN mismatch(%s), controllerUrl(%s).\n\r",
15:            "_ecs_verifySsl",
16:            125LL,
17:            cert_subject_name,
18:            global_controller_host);
19:        if ( qword_5CCB8 )
20:        {
21:
22:                                                          s).\n\r");
23:
24:
25:
26:
27:    }
     //
   }
```

"Let's eat, Grandma!"

vs

"Let's eat Grandma!"

The broken chain of trust — Cloud (CVE-2025-9291)

```
{
   "id" : 1,
   "method" : "helloCloud",
   "params" : {
      "alias" : "ER605",
      "authCode" : "[REDACTED]",
      "cloudUserName" : "",
      "controllerVersion" : "",
      "deviceHwVer" : "2.0",
      "deviceId" : "[REDACTED]",
      "deviceMac" : "[REDACTED]",
      "deviceModel" : "ER605",
      "deviceName" : "ER605",
      "deviceType" : "SMBROUTER",
      "fwId" : "",
      "fwVer" : "2.2.6 Build 20240718 Rel.82712",
      "hwId" : "[REDACTED]",
      "oemId" : "[REDACTED]",
      "tcspVer" : "1.2"
   }
}

{
   "error_code" : 0,
   "id" : 1,
   "result" : {
      "cachedSvr" : "n-euw1-device-omada.tplinkcloud.com:443",
      "illegalType" : 0,
      "validTimeOnDevice" : 86400
   }
}
```

```
__int64 ecs_verifySsl(unsigned int preverify_ok, void* store_ctx) {
    // ...
01:   if ( !error_depth )
02:    {
03:      subject_name_str = strstr(subject_name_buf, "/CN=");
04:      if ( !subject_name_str )
05:        return 0;
06:      cert_subject_name = subject_name_str + 4;
07:      v11 = strchr(subject_name_str + 4, '/');
08:      if ( v11 )
09:        *v11 = 0;
10:      if ( strstr(global_controller_host, "tplinkcloud.com") && !strstr(cert_subject_name, "tplinkcloud.com") )
11:      {
12:        if ( HIDWORD(qword_5CCB8) )
13:          printf(
14:            "[ECS][ERROR]%s():%5d @ verify error:CN mismatch(%s), controllerUrl(%s).\n\r",
15:            "_ecs_verifySsl",
16:            125LL,
17:            cert_subject_name,
18:            global_controller_host);
19:        if ( qword_5CCB8 )
20:        {
21:          // ...
22:          ecs_log(2LL, "[ECS][ERROR]<%s>%s():%5d @ verify error:CN mismatch(%s), controllerUrl(%s).\n\r");
23:          // ...
24:        }
25:        return 0;
26:      }
27:    }
    // ...
```

We can use a public IP to bypass the CN check :-}

Client enumeration (FSCT-2025-003, FSCT-2025-011)

- When adopting a Client via Cloud all you need to know is the S/N, no proof-of-ownership required
- S/N are sequential, can be retrieved using Omada Cloud API; Client info, **such as model and MAC**, can be inferred from its S/N
- **VERY susceptible to brute-forcing**

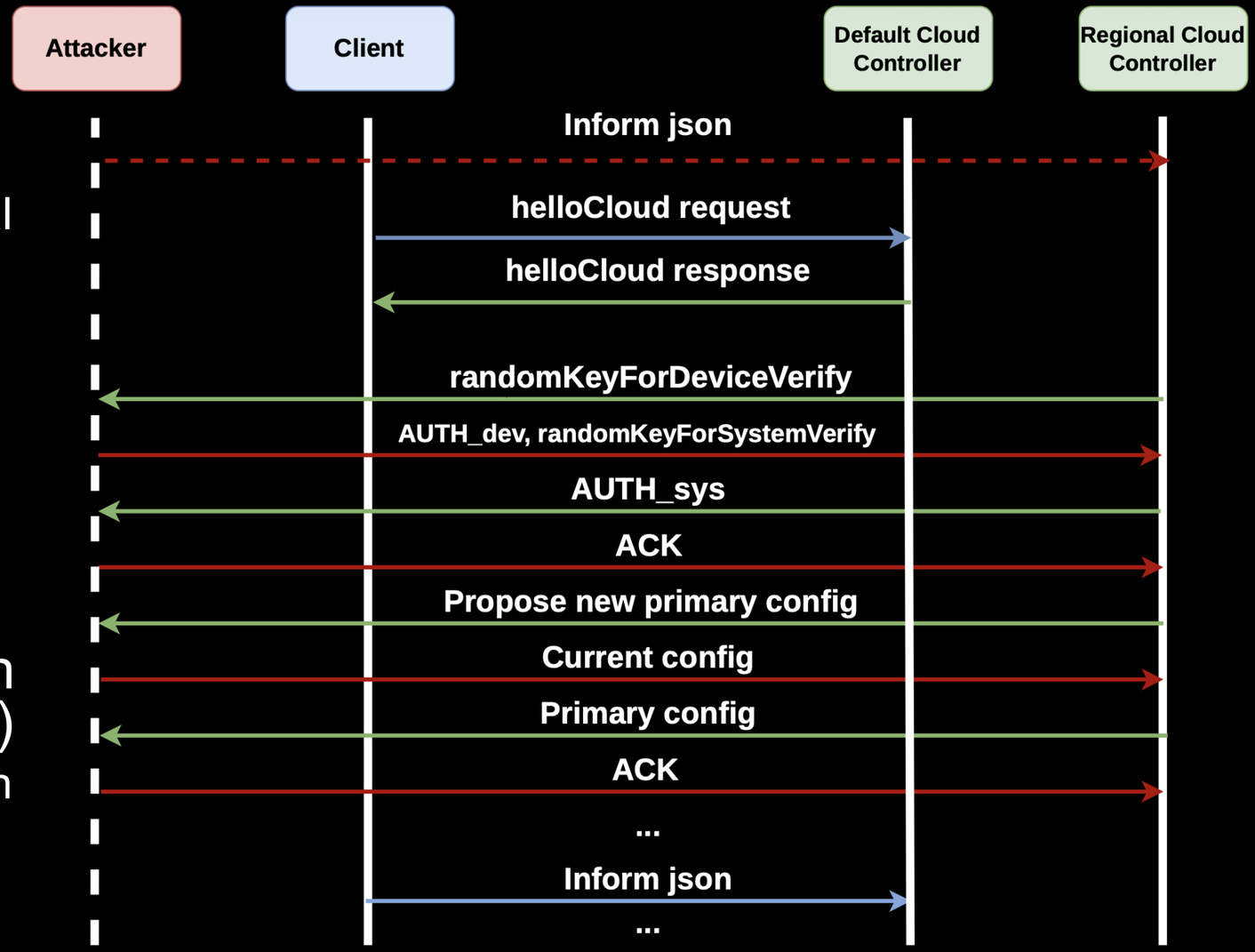
INDEX	SN CODE	NAME	STATUS	RESULT
1	22[redacted]1246	A8-6E-84-[redacted]	Offline	Failed to adopt this gateway because a gateway already exists in this site.
2	22[redacted]1245	A8-6E-84-[redacted]	Online	Failed to adopt this gateway because a gateway already exists in this site.
3	22[redacted]1244	A8-6E-84-[redacted]	Offline	Failed to adopt this gateway because a gateway already exists in this site.
4	22[redacted]1247	A8-6E-84-[redacted]	Offline	

Client hijacking, Cloud Controllers (CVE-2025-15630)

- Client contacts the Default Controller (Discovery phase)
- Default Controller sends the URL of a Regional Controller (Adoption and other phases kick in)

- **Discovery and Adoption phases are not bound to the same ”state machine”**

- **Attackers may start the Adoption phase on Client’s behalf** (don’t even need the Discovery request to arrive!)
  - A “fake” Client will get adopted and appear in the Web UI
  - You can spam Cloud Controllers with fake Clients

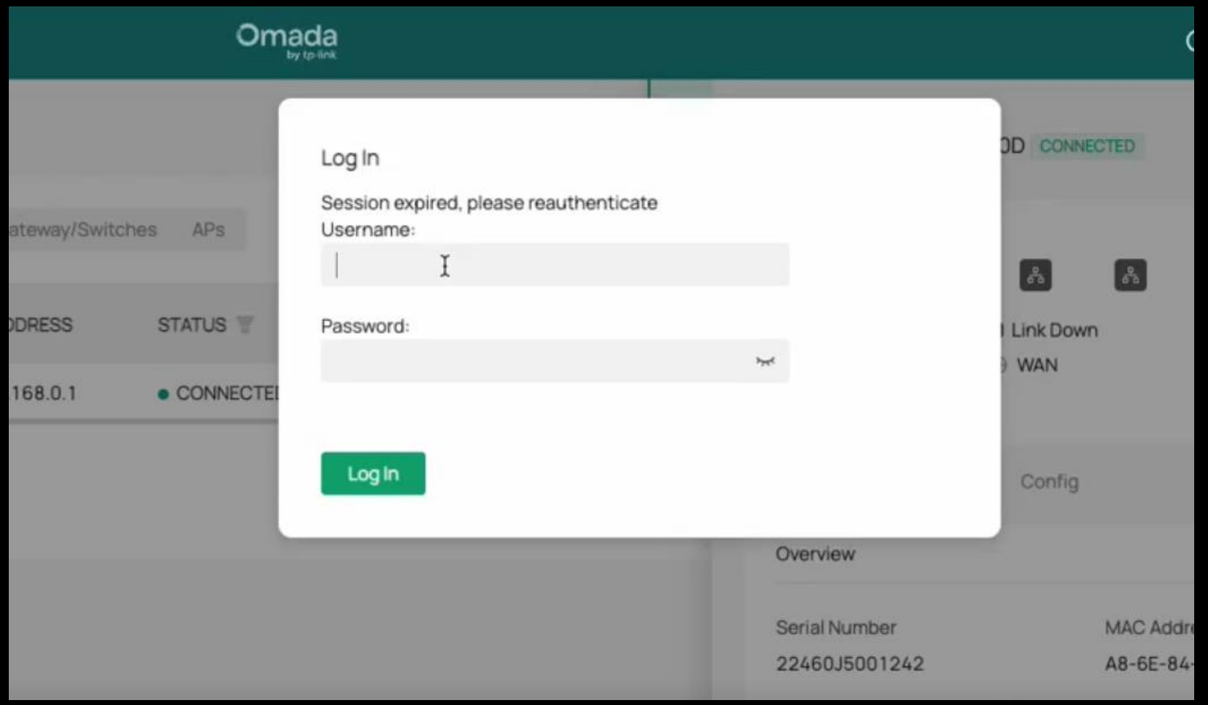


Stored XSS in Controller Web UI (CVE-2025-9289)

- The properties of adopted Clients are displayed in the Web UI
- Older version jQuery, calls “eval()” to update the UI with Client’s information

```json
{"header":{"version":"2.2.0","mac":"A8:6E:84:XX:XX:XX","type":
    1048580,"device":"gateway","error":0,"dest":"","verCap":3},
    "body":{"devCap":{"supportIPsecFailover":1,"specification":
    { ..."fwVer":"XXX<script>alert(1)</script>" ... }}}}
```

- We couldn’t do much because of restrictive content security policy (CSP)




```
Content-Security-Policy: default-src 'self' https://*.tplinkcloud.com/;script-src
'self' 'unsafe-eval' 'sha256-7W9UiBaYGlOHpT1aQBLegqffUVHbYq6/ZAb+ErjUb40=' 'sha256-
VGQ8jNTL2g0e8wPwOgyCQJDqhuRgfV7gRYexcBkBe4Y=' 'sha256-x2jgB1zBLi30IsfY+
VNgWjwBGeHPJxOSrzl+IdsT6k0=' 'sha256-0AHZXO4clnpdcxqdmASPBEp4JCIrtaxIX/mUuL1kzZw='
'sha256-lfXlPY3+MCPOPb4mrw1Y961+745U3WlDQVcOXdchSQc=';style-src 'self' 'unsafe-
inline';connect-src 'self' https://*.tplinkcloud.com/ https://*.tplinkcloud.com:
8843/ wss://*.tplinkcloud.com/ https://*.tiles.mapbox.com https://api.mapbox.com
https://events.mapbox.com ;frame-src 'self' data:;img-src 'self' https://*.
tplinkcloud.com/ https://*.mzstatic.com/ https://play-lh.googleusercontent.com/
data: blob:;child-src blob: ;worker-src blob: ;object-src 'self' data: blob:
```

Cross-Origin Resource Sharing (CORS) bypass (CVE-2025-9292)

- We checked the CSP of Cloud Controllers
- The Omada Cloud runs on AWS

- There are bits of the default CSP that allow Cross-Site requests to anything hosted on AWS…

default-src 'self' https://*.tplinkcloud.com/; script-src 'self' 'unsafe-eval' https://www.paypal.com/ https://www.paypalobjects.com/ https://js.stripe.com 'sha256-7W9UiBaYGlOHpT1aQBLegqffUVHbYq6/ZAb+ErjUb40=' 'sha256-VGQ8jNTL2g0e8wPwOgyCQJDqhuRgfV7gRYexcBkBe4Y=' 'sha256-+9dQLByJ0rMH7ojZkdfnL0p0S0pZqKxWTlh7vSa+FMg=' 'sha256-x2jgB1zBLi30IsfY+VNgWjwBGeHPJxOSrzl+ldsT6k0=' 'sha256-0AHZXO4clnpdcxqdmASPBEp4JClrtaxlX/mUuL1kzZw=' 'sha256-lfXlPY3+MCPOPb4mrw1Y961+745U3WlDQVcOXdchSQc=' 'sha256-VohL58KrXs+2LMTip+0SXk2/JwaZNxG/j9hadlPKc2E='; style-src 'self' 'unsafe-inline'; connect-src 'self' https://*.cloudfront.net/ https://*.tplinkcloud.com/ https://*.tplinkcloud.com:8843/ http://*.tplinkcloud.com:8088/ wss://*.tplinknbu.com/ ws://localhost:*/ wss://*.tplinkcloud.com/ https://*.paypal.com/ https://api.stripe.com https://*.tiles.mapbox.com https://api.mapbox.com/ https://events.mapbox.com https://*.amazonaws.com/ data:; frame-src 'self' OmadaSightWebPlayer: vigiwebplayer: https://*.tplinkcloud.com/ https://js.stripe.com https://hooks.stripe.com https://*.paypal.com/ data: blob:; img-src 'self' https://*.cloudfront.net/ https://*.tplinkcloud.com/ https://*.amazonaws.com/ https://*.mzstatic.com/ https://play-lh.googleusercontent.com/ https://kcart.alipay.com/ data: blob:; child-src blob:; worker-src https://*.tplinkcloud.com blob:; media-src https://*.tplinkcloud.com/ https://*.cloudfront.net/ https://*.amazonaws.com/ blob:; object-src 'self' data: blob:

Hippity hoppity your network is our property

Local compromise

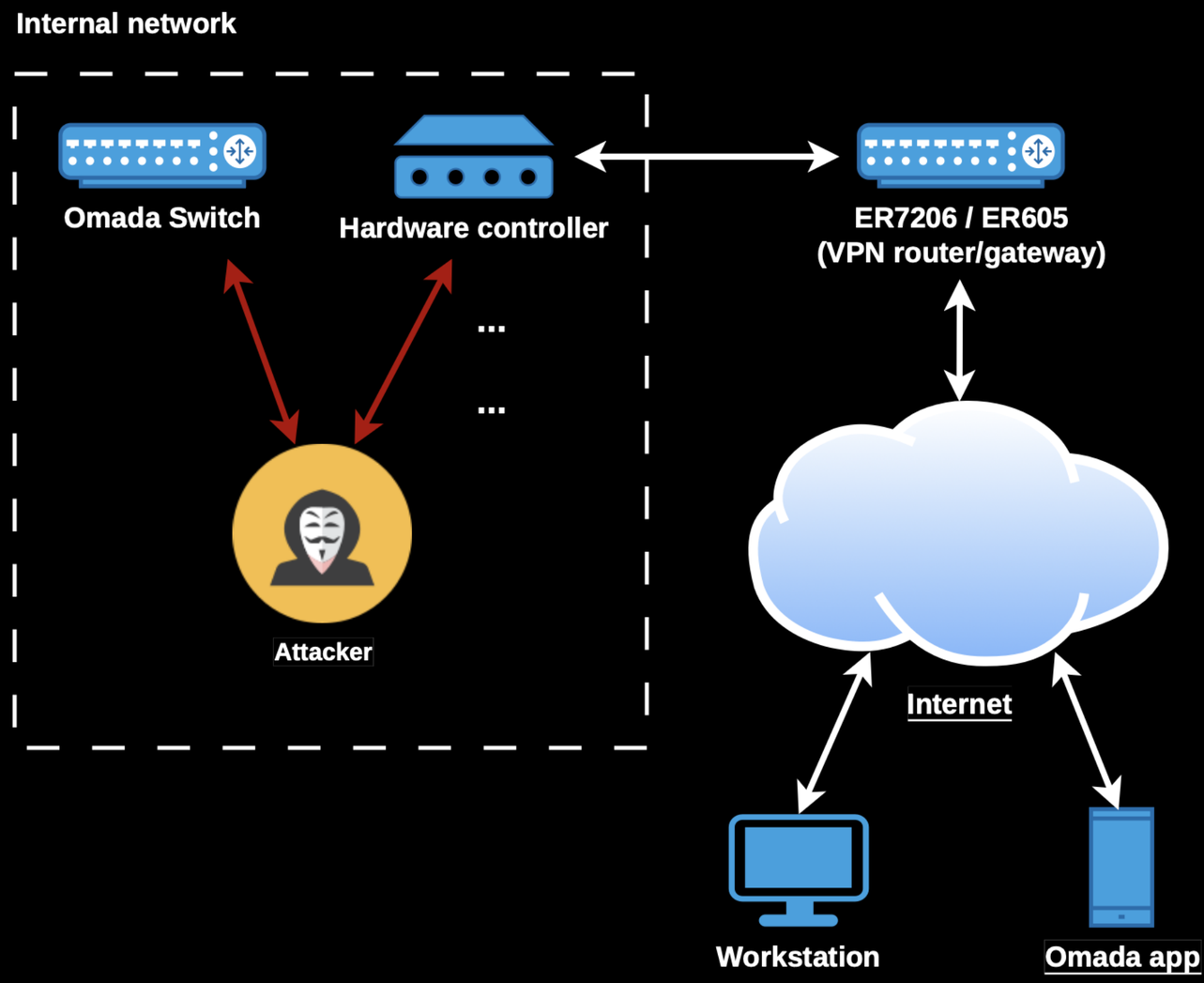
ATTACK SCENARIO:

The attacker is positioned inside of the victim's local network:

1. Intercept the Discovery request (UDP broadcast) and respond as a fake Controller
2. Forward the Discovery request (modified) to the real Controller, posing as a fake Client
3. The Controller responds to the attacker, who can now Intercept, modify, and forward all comms between the real Controller and the real Client (CVE-2025-15628, CVE-2025-15544 or CVE-2025-9290)

IMPACT: attacker can

- Retrieve and modify sensitive information (e.g., device configuration)
- Take over Clients



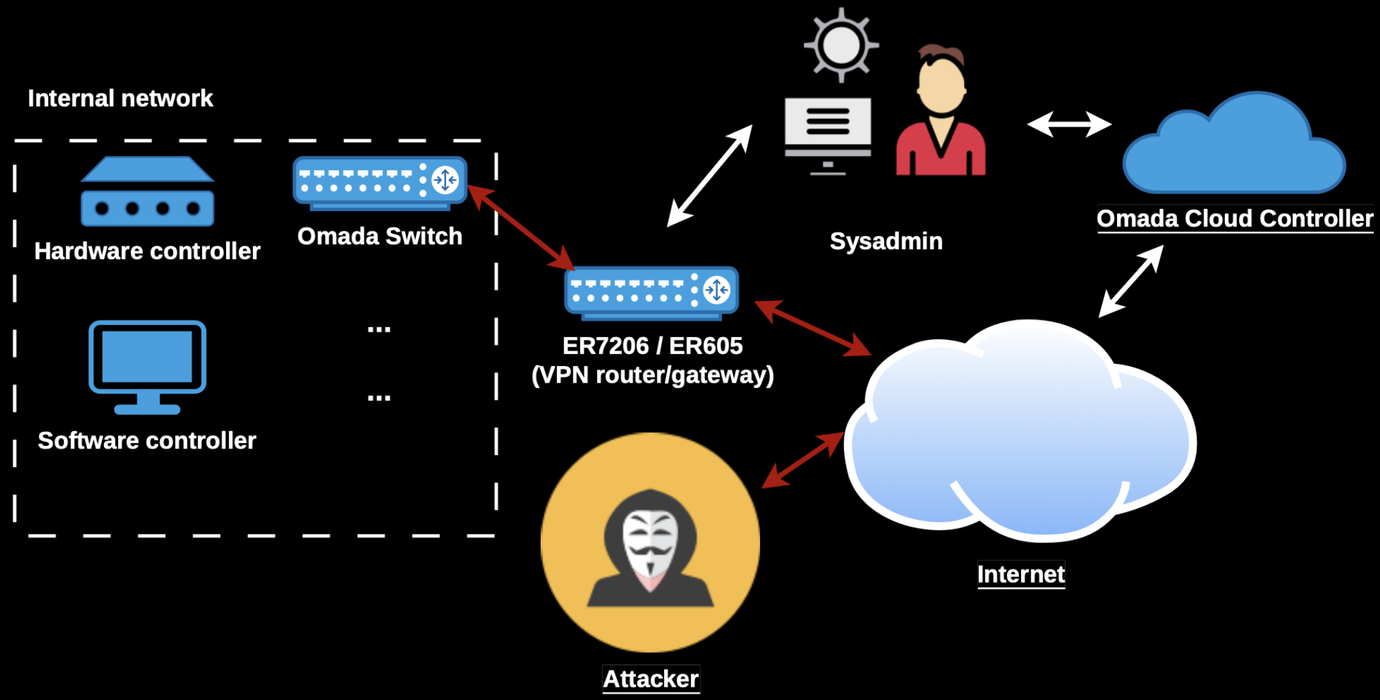
```
(venv) standash@moria:~/stuff/vr/tplink/xploits$
```

External compromise

ATTACK SCENARIO:

The attacker is positioned outside victim's network, and exploits the race-condition:

1. Collect MAC addresses (Omada API or just brute-force) to impersonate a not-yet-adopted Client
2. Resend the message every 60 seconds. To target 1000 MACs attacker only needs ~17 requests per second -> wait for a device "adoption"
3. Obtain original device provisioned configuration
4. Exploit the stored XSS/CSP bypass to phish admins and potentially steal Cloud account credentials

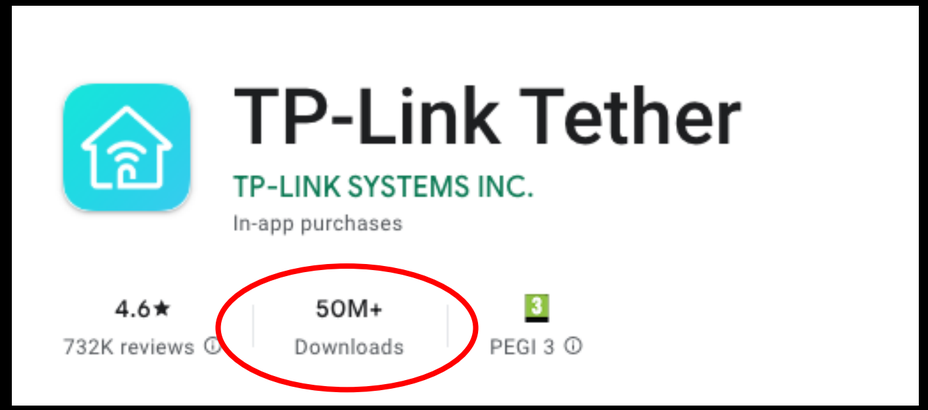


IMPACT: attacker can

- Retrieve sensitive information (hashed site-credentials, VPN keys, etc.)
- Through stolen cloud credentials: modify device configurations, modify firewall rules, add VPNs to access internal networks, obtain "site-credentials" from site settings…

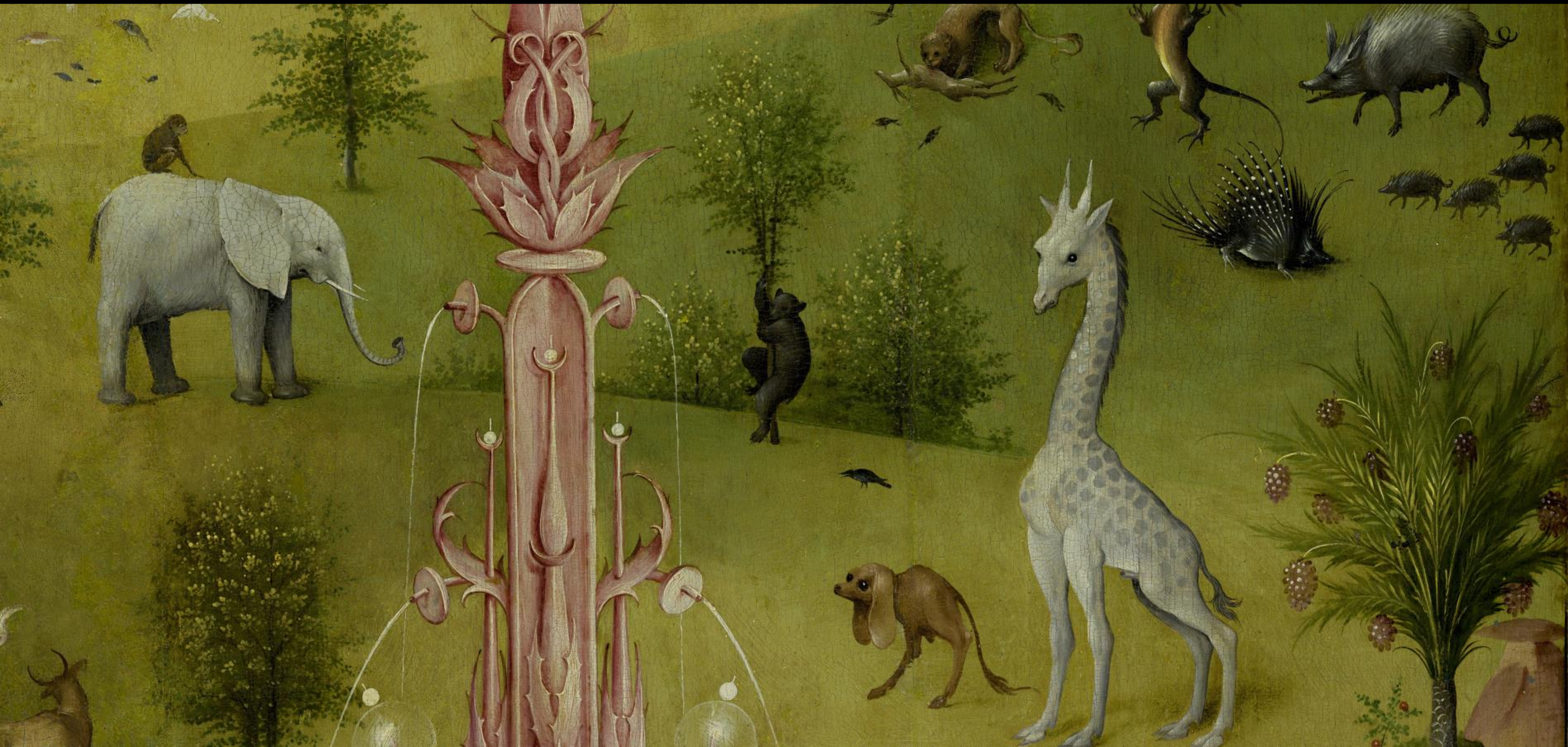
The chain of trust issue is way worse (CVE-2025-9293)

- The same chain of trust was used in lots of other TP-Link product families
  - Omada, Festa, Tapo, Kasa, VIGI, Android apps…
  - E.g. we could sniff credentials from Android apps TLS connections…

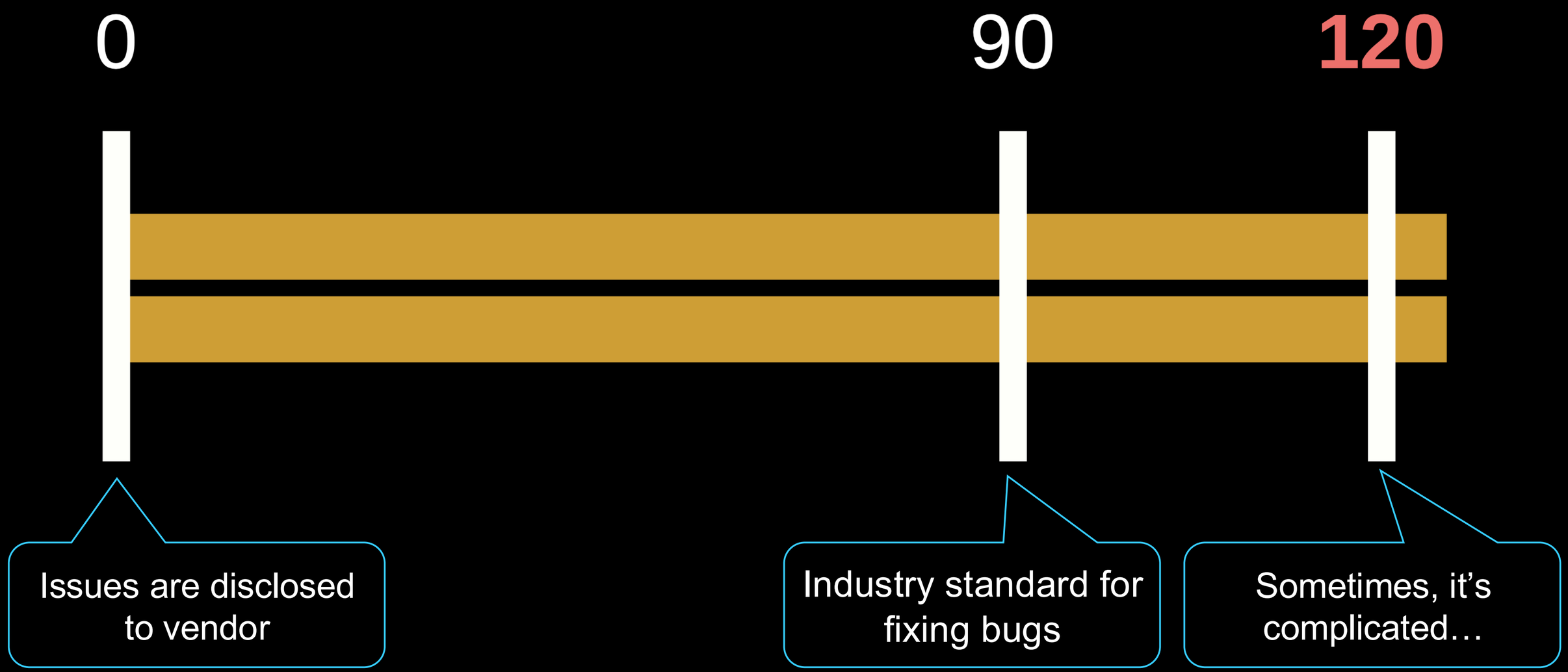


Aginet	**2.11.23**
Deco	3.9.76
Festa	1.6.9
Kasa	3.4.101
KidShield	1.1.19
Omada	4.24.13
Omada Guard	1.0.14
Tapo	3.11.114
Tether	4.10.42
tpCamera	3.2.12
VIGI	2.7.16
Wi-Fi Navi	1.4.8
WiFi Toolkit	1.4.2

Disclosure and some takeaways



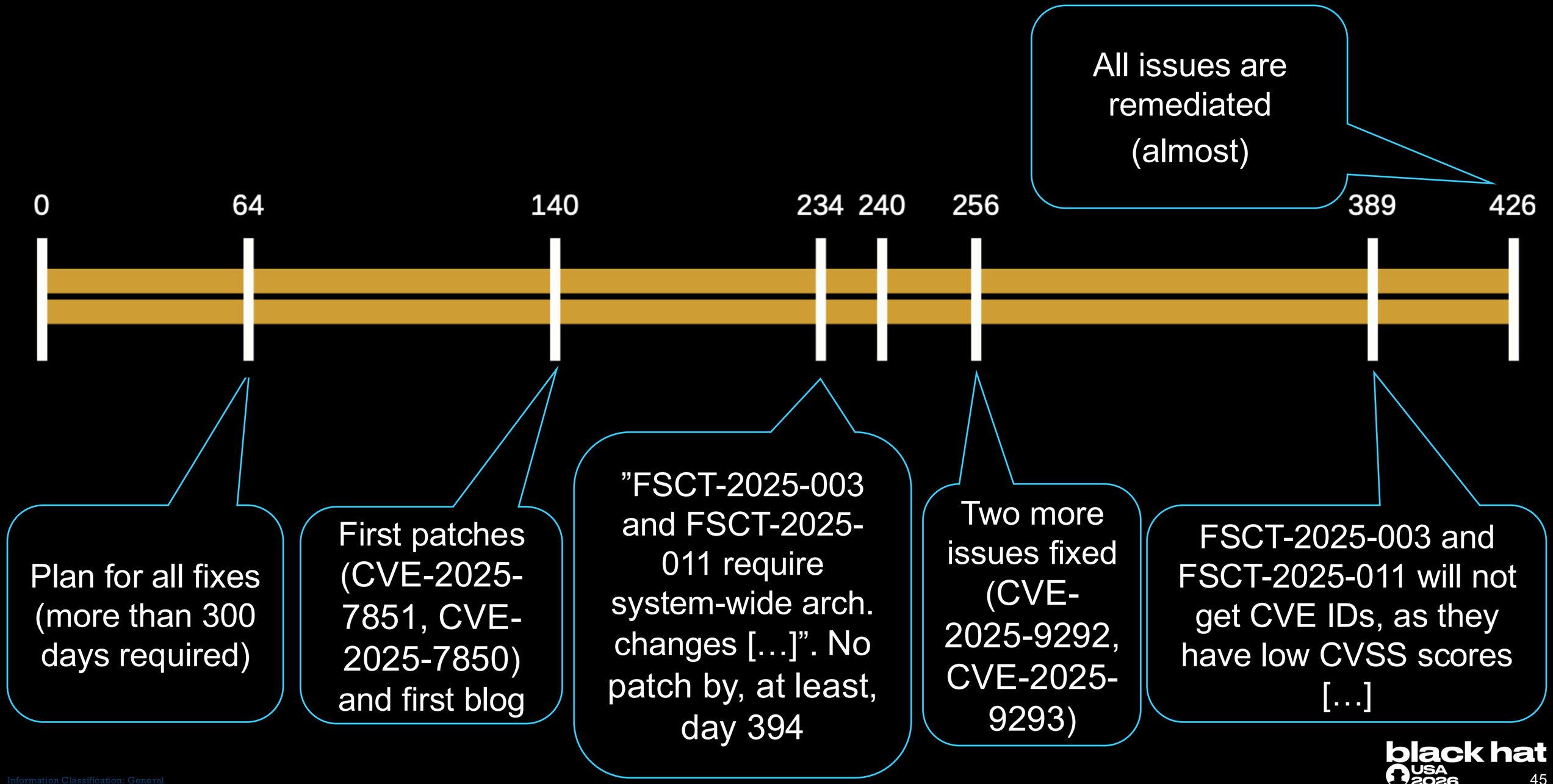
Disclosure timeline (typical)



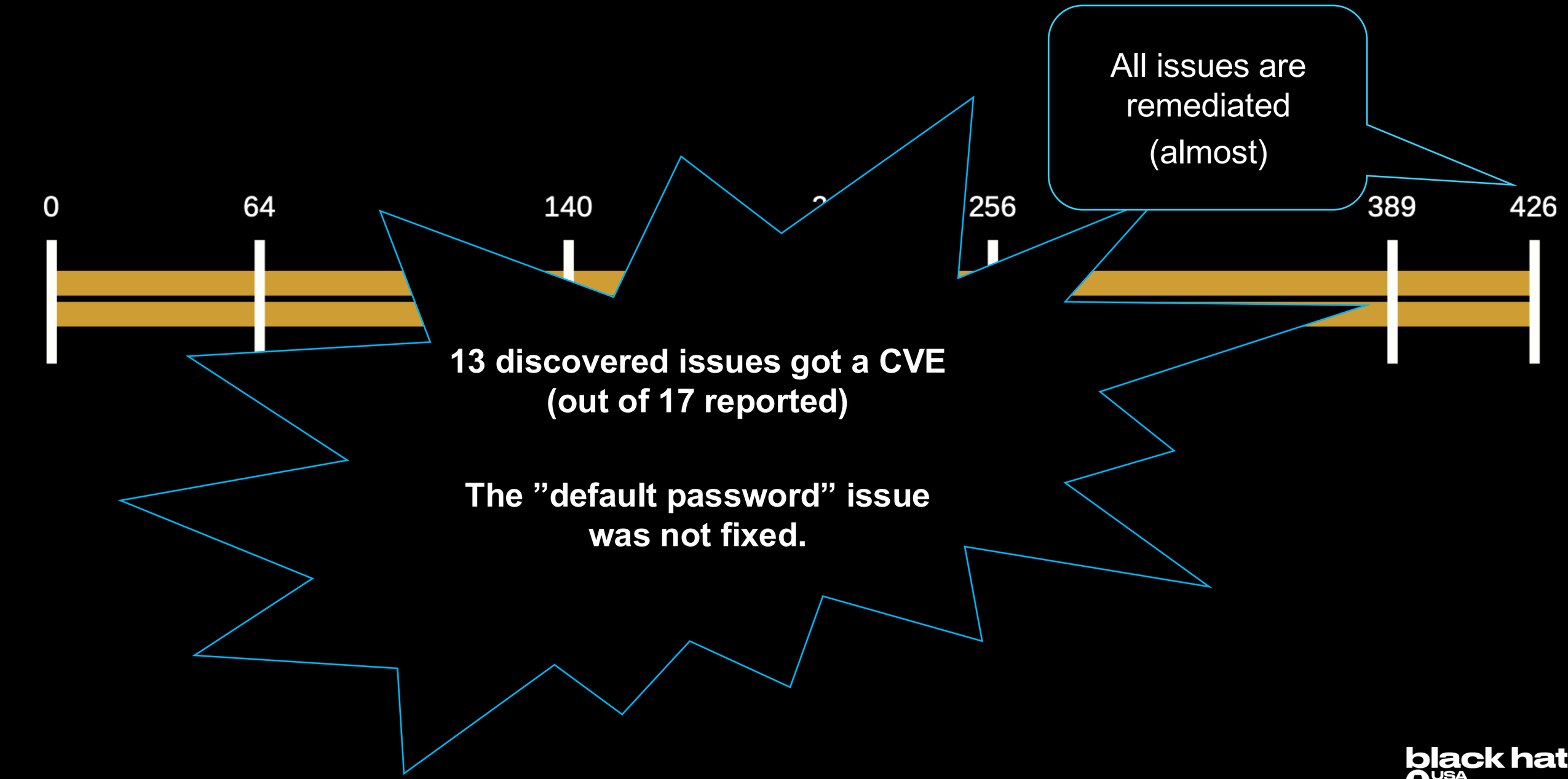
Disclosure timeline (this research)

This is how long it took this time!

Disclosure timeline (this research)



Disclosure timeline (this research)



Takeaways

- For vendors:
  - Cryptography is engineering and not a creative process – follow standards, use proven algorithms, plan your PKI ahead
  - Security through obscurity never works
  - Bad design choices can cause serious problems later – e.g. shared vulns affecting many product families: **very loooong patch windows -> users are exposed all this time**

- For users:
  - If you use any ZTP platforms, enable every possible security measure such as MFA, don't use a default password, rotate passwords & keys frequently, update your devices!

https://www.forescout.com/research-labs-overview/